ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA



LG DUT CYBER SECURITY LAB

BÁO CÁO THỰC TẬP TUẦN 3

Triển khai các phương thức truyền thông Unicast, Broadcast, Multicast và Anycast trên IPv4 và IPv6

Thành viên Nhóm 4

1. Nguyễn Trường Bảo
2. Mai Thị Mỹ Đình
3. Bùi Phương Chi
4. Nguyễn Đình Nhất
5. Đỗ Hữu Dũng
6. Nguyễn Thành Lưu

Đà Nẵng, ngày . . . tháng . . . năm . . .

Mục lục

1 Giới thiệu **1**

2 Yêu cầu thực nghiệm **1**

3 Cơ sở lý thuyết **1**

3.1 Unicast . . . 2
3.2 Broadcast . . . 3
3.3 Multicast . . . 4
3.4 Anycast . . . 5
3.5 Socket . . . 5
3.6 Vai trò của tường lửa trong kiểm soát lưu lượng . . . 6

4 Mô hình triển khai **6**

4.1 Vai trò và địa chỉ IP của thiết bị . . . 7
4.2 Vai trò của Windows 2 . . . 7
4.3 Chương trình Python gửi và nhận . . . 7
4.3.1 IPv4 Unicast . . . 7
4.3.2 IPv4 Broadcast . . . 8
4.3.3 IPv4 Multicast . . . 9
4.3.4 IPv6 Unicast . . . 10
4.3.5 IPv6 Multicast . . . 11
4.3.6 IPv6 Anycast . . . 12

5 Triển khai và đánh giá kết quả **12**

5.1 Trạng thái trước khi áp dụng rule tường lửa . . . 13
5.1.1 IPv4 Unicast . . . 13
5.1.2 IPv4 Broadcast . . . 13
5.1.3 IPv4 Multicast . . . 14
5.1.4 IPv6 Unicast . . . 16
5.1.5 IPv6 Anycast . . . 17
5.1.6 IPv6 Multicast . . . 19
5.2 Trạng thái sau khi áp dụng rule tường lửa . . . 21
5.2.1 IPv4 Unicast – **BỊ CHẶN** . . . 21
5.2.2 IPv4 Broadcast – **ĐƯỢC CHO PHÉP** . . . 23
5.2.3 IPv4 Multicast – **ĐƯỢC CHO PHÉP** theo nhóm . . . 24
5.2.4 IPv6 Unicast – **BỊ CHẶN** . . . 25

5.2.5 IPv6 Multicast – chỉ nhóm được khai báo mới đi qua 26
5.2.6 IPv6 Anycast – **ĐƯỢC CHO PHÉP** 28
5.3 Bảng tổng hợp so sánh 31

6 Đánh giá hiệu năng và an toàn **32**

7 Kết luận **32**

Tài liệu tham khảo **32**

1 Giới thiệu

Trong chủ đề này, nhóm tập trung nghiên cứu và triển khai các phương thức truyền thông phổ biến trong mạng máy tính, bao gồm Unicast, Broadcast, Multicast và Anycast trên nền tảng giao thức IPv4 và IPv6. Mục tiêu là tìm hiểu nguyên lý hoạt động của từng phương thức, đồng thời đánh giá sự khác biệt về cơ chế truyền dữ liệu, hiệu năng và khả năng ứng dụng trong các hệ thống mạng hiện đại.

Để phục vụ quá trình thực nghiệm, nhóm xây dựng một mô hình mạng gồm ba thiết bị, trong đó có Raspberry Pi 4 cài đặt WebOS và các máy trạm Windows, được kết nối thông qua một mạng LAN riêng biệt. Mạng thử nghiệm sử dụng dải địa chỉ IP độc lập với mạng LAB nhằm tránh xung đột địa chỉ, tạo môi trường kiểm thử ổn định và không ảnh hưởng đến các hệ thống khác.

Trên mô hình này, nhóm phát triển các chương trình bằng ngôn ngữ Python để gửi và nhận dữ liệu giữa các thiết bị. Đối với IPv4, chương trình hỗ trợ ba phương thức Unicast, Broadcast và Multicast; trong đó Multicast sử dụng IGMP (Internet Group Management Protocol) để quản lý việc tham gia và rời khỏi nhóm nhận dữ liệu.

Đối với IPv6, nhóm triển khai các chương trình tương tự cho Unicast, Multicast và Anycast. Do IPv6 không hỗ trợ Broadcast, Multicast được sử dụng thay thế trong các trường hợp cần truyền dữ liệu đến nhiều thiết bị cùng lúc. Anycast cho phép gói tin được định tuyến đến nút có đường đi tối ưu trong số các thiết bị cùng chia sẻ một địa chỉ.

Sau khi hoàn thành triển khai, nhóm đánh giá từng phương thức theo các tiêu chí: tốc độ truyền dữ liệu, độ trễ, độ ổn định, khả năng mở rộng và mức độ an toàn. Kết quả thực nghiệm được dùng để so sánh ưu điểm, hạn chế của từng phương thức cũng như sự khác biệt giữa IPv4 và IPv6, từ đó đề xuất phương án phù hợp cho các hệ thống mạng thực tế.

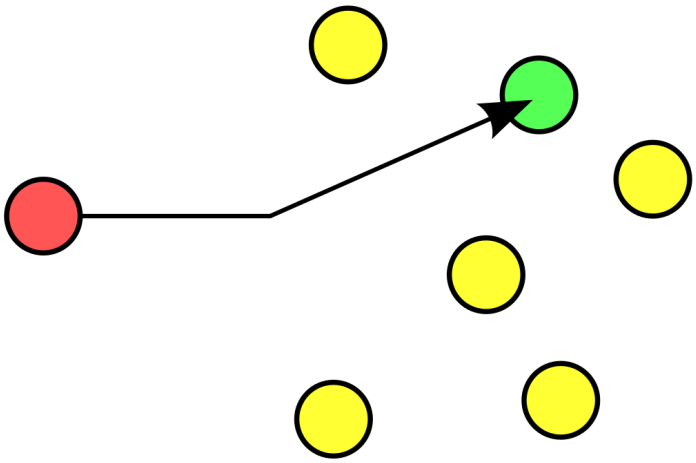
2 Yêu cầu thực nghiệm

Để hoàn thành nội dung thực nghiệm, nhóm thực hiện các yêu cầu sau:

- Xây dựng mô hình mạng gồm Raspberry Pi 4 cài đặt WebOS và các máy trạm Windows, kết nối qua một mạng LAN riêng biệt.
- Cấu hình hệ thống mạng IPv4 và IPv6, bảo đảm các thiết bị giao tiếp ổn định trong môi trường thử nghiệm.
- Phát triển chương trình Python gửi và nhận dữ liệu theo các phương thức Unicast, Broadcast và Multicast trên IPv4.
- Phát triển chương trình Python gửi và nhận dữ liệu theo các phương thức Unicast, Multicast và Anycast trên IPv6.
- Triển khai IGMP để quản lý nhóm Multicast trong IPv4 và kiểm tra quá trình tham gia, rời khỏi nhóm của thiết bị.
- Xây dựng kịch bản kiểm thử cho từng phương thức truyền thông.
- So sánh IPv4 và IPv6 về cơ chế truyền dữ liệu, phạm vi truyền thông, khả năng mở rộng và hiệu quả sử dụng băng thông.
- Đánh giá tốc độ, độ trễ, độ ổn định, mức độ an toàn và khả năng kiểm soát truy cập.
- Đề xuất phương thức truyền phù hợp cho từng bài toán dựa trên kết quả thực nghiệm.

3 Cơ sở lý thuyết

3.1 Unicast



Hình 1: Mô hình truyền thông Unicast: một nguồn gửi đến một đích.

Unicast là phương thức truyền thông trong đó một thiết bị nguồn gửi dữ liệu đến duy nhất một thiết bị đích. Đây là phương thức phổ biến nhất trong các mạng IP và được sử dụng trong hầu hết ứng dụng như truy cập website, truyền tệp, thư điện tử, đăng nhập từ xa (SSH) và kết nối máy khách–máy chủ.

Đối với IPv4, Unicast sử dụng một địa chỉ IPv4 duy nhất làm địa chỉ đích, chẳng hạn `192.168.1.10`. Gói tin chỉ được chuyển đến thiết bị sở hữu địa chỉ này. Đối với IPv6, nguyên lý hoạt động tương tự nhưng sử dụng địa chỉ 128 bit, chẳng hạn `2001:db8:85a3::8a2e:370:7334`; không gian địa chỉ lớn giúp định danh thiết bị hiệu quả và hỗ trợ mở rộng mạng.

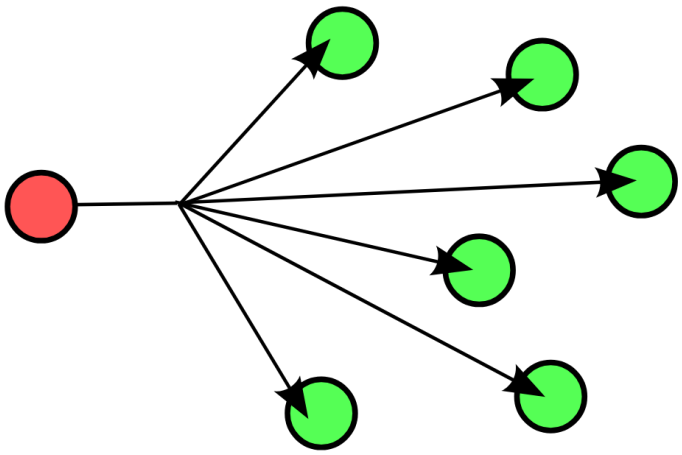
Nguyên lý hoạt động

1. Thiết bị nguồn tạo gói tin và gắn địa chỉ IP của thiết bị đích.
2. Gói tin được truyền qua mạng.
3. Router tra cứu bảng định tuyến để xác định đường đi và chuyển tiếp gói tin.
4. Thiết bị đích nhận, kiểm tra và xử lý dữ liệu.

Đặc điểm

- Một gói tin chỉ được gửi đến một thiết bị nhận.
- Dễ quản lý, kiểm soát truy cập và phù hợp với giao tiếp một–một.
- Tính riêng tư cao hơn Broadcast.
- Khi cùng một nội dung cần gửi đến nhiều thiết bị, nguồn phải tạo nhiều bản sao nên tiêu tốn băng thông.

3.2 Broadcast



Hình 2: Mô hình truyền thông Broadcast: một nguồn gửi đến toàn bộ thiết bị trong miền quảng bá.

Broadcast là phương thức trong đó một thiết bị gửi dữ liệu đến tất cả thiết bị trong cùng một miền quảng bá (Broadcast Domain). Broadcast thường được sử dụng trong các giao thức khám phá thiết bị hoặc cấp phát địa chỉ như ARP và DHCP.

Trong IPv4, Broadcast sử dụng địa chỉ quảng bá của mạng con. Ví dụ, với mạng `192.168.1.0/24`, địa chỉ Broadcast là `192.168.1.255`; mọi thiết bị trong cùng subnet đều nhận được gói tin gửi đến địa chỉ này.

IPv6 không hỗ trợ Broadcast. Thay vào đó, IPv6 sử dụng Multicast cho các chức năng tương tự; chẳng hạn `ff02::1` đại diện cho tất cả nút trong cùng liên kết (All Nodes Multicast). Việc loại bỏ Broadcast giúp giảm lưu lượng quảng bá không cần thiết và tăng hiệu quả mạng.

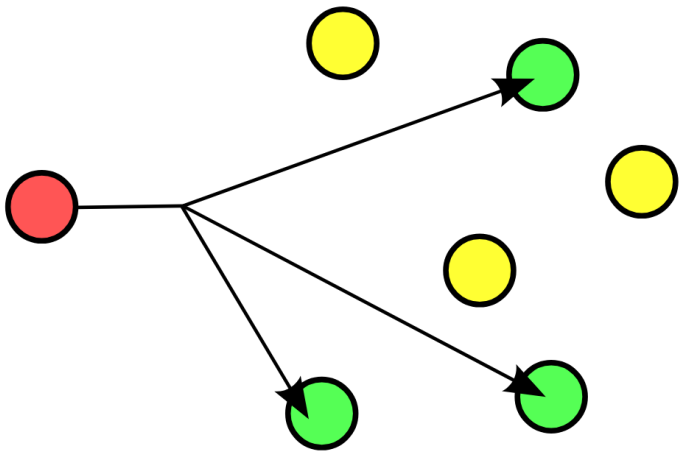
Nguyên lý hoạt động trên IPv4

1. Thiết bị nguồn gửi gói tin đến địa chỉ Broadcast của mạng.
2. Khung Ethernet sử dụng địa chỉ MAC đích `FF:FF:FF:FF:FF:FF`.
3. Switch chuyển tiếp gói tin đến tất cả cổng trong cùng Broadcast Domain.
4. Mỗi thiết bị quyết định tiếp nhận hoặc bỏ qua dữ liệu tùy theo giao thức và dịch vụ.

Đặc điểm

- Một gói tin được gửi đến toàn bộ thiết bị trong cùng mạng.
- Phù hợp với cơ chế khám phá thiết bị và cấu hình mạng tự động.
- Có thể làm tăng lưu lượng nếu bị sử dụng quá mức.
- Có nguy cơ bị lợi dụng để quét mạng, tấn công từ chối dịch vụ hoặc tạo Broadcast Storm.

3.3 Multicast



Hình 3: Mô hình truyền thông Multicast: nguồn chỉ gửi đến các thành viên của nhóm.

Multicast là phương thức trong đó một thiết bị nguồn gửi dữ liệu đến một nhóm thiết bị đã đăng ký nhận. So với Unicast, Multicast tiết kiệm băng thông khi cùng một dữ liệu cần truyền đến nhiều thiết bị; đồng thời khắc phục hạn chế của Broadcast bằng cách chỉ chuyển dữ liệu đến các thiết bị quan tâm.

Trong IPv4, địa chỉ Multicast thuộc dải `224.0.0.0/4`, từ `224.0.0.0` đến `239.255.255.255`. IGMP được dùng để quản lý thiết bị tham gia nhóm. Trong IPv6, địa chỉ Multicast thuộc dải `ff00::/8`; MLD (Multicast Listener Discovery), xây dựng trên ICMPv6, quản lý quá trình tham gia và rời khỏi nhóm.

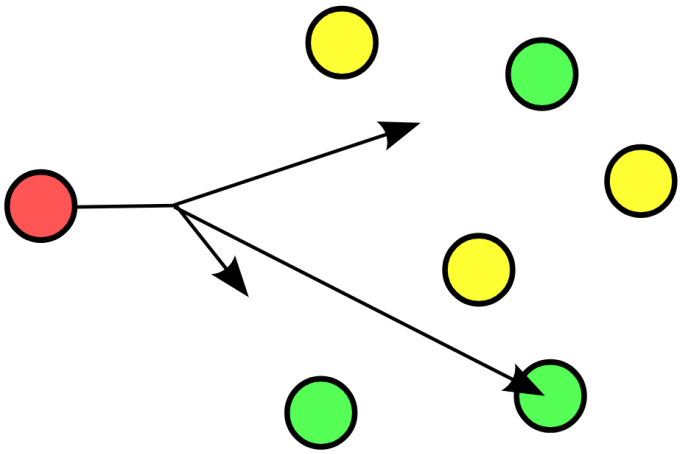
Nguyên lý hoạt động

1. Thiết bị nguồn gửi gói tin đến một địa chỉ Multicast.
2. Thiết bị muốn nhận dữ liệu phải tham gia nhóm tương ứng.
3. Router và switch chỉ chuyển tiếp gói tin đến các nhánh có thành viên của nhóm.
4. Thiết bị không thuộc nhóm sẽ không nhận dữ liệu.

Đặc điểm

- Tiết kiệm băng thông khi truyền cùng dữ liệu đến nhiều thiết bị.
- Chỉ thành viên trong nhóm mới nhận được dữ liệu.
- Giảm lưu lượng mạng so với Broadcast.
- Cần IGMP Snooping hoặc MLD Snooping để tối ưu chuyển tiếp tại switch.
- Cần cơ chế quản lý quá trình tham gia và rời khỏi nhóm.

3.4 Anycast



Hình 4: Mô hình truyền thông Anycast: dữ liệu được chuyển đến nút có đường định tuyến tối ưu.

Anycast là phương thức trong đó nhiều thiết bị cùng sử dụng một địa chỉ IP, nhưng gói tin chỉ được chuyển đến một thiết bị đích, thường là nút có đường định tuyến tối ưu nhất từ phía nguồn. Phương thức này giúp giảm độ trễ, cân bằng tải và tăng tính sẵn sàng của dịch vụ.

Anycast không có dải địa chỉ riêng. Các địa chỉ Unicast thông thường được cấu hình trên nhiều thiết bị và được hệ thống định tuyến xử lý như một địa chỉ Anycast. Phương thức này được sử dụng rộng rãi trong DNS, CDN và các dịch vụ phân tán trên Internet.

Nguyên lý hoạt động

1. Thiết bị nguồn gửi gói tin đến địa chỉ Anycast.
2. Router tra cứu bảng định tuyến để xác định đường đi tối ưu.
3. Thiết bị có chi phí định tuyến thấp nhất được lựa chọn nhận gói tin.
4. Nếu nút đang phục vụ gặp sự cố, hệ thống định tuyến chuyển lưu lượng đến nút khác cùng sử dụng địa chỉ Anycast.

Đặc điểm

- Nhiều thiết bị cùng chia sẻ một địa chỉ IP; phía gửi không cần biết máy chủ cụ thể nào sẽ nhận yêu cầu.
- Giảm độ trễ, hỗ trợ phân phối tải và tăng tính liên tục của dịch vụ.
- Phụ thuộc vào cơ chế định tuyến như OSPF, IS-IS hoặc BGP.
- Khái niệm “gần nhất” được hiểu theo chi phí định tuyến, không nhất thiết theo khoảng cách địa lý.

3.5 Socket

Socket là cơ chế lập trình mạng cho phép các tiến trình trên cùng một thiết bị hoặc trên các thiết bị khác nhau thiết lập kết nối và trao đổi dữ liệu thông qua địa chỉ IP và số hiệu cổng. Socket đóng vai trò là giao diện giữa chương trình ứng dụng và tầng giao vận của mô hình TCP/IP.

Trong Python, thư viện `socket` hỗ trợ xây dựng ứng dụng trên cả IPv4 và IPv6 với hai giao thức giao vận phổ biến:

- **TCP** là giao thức hướng kết nối, bảo đảm dữ liệu được truyền đầy đủ, đúng thứ tự và có cơ chế kiểm tra lỗi. TCP phù hợp với truyền tệp, truy cập web và điều khiển từ xa.
- **UDP** là giao thức không hướng kết nối, không yêu cầu thiết lập phiên và không bảo đảm giao nhận. Bù lại, UDP có độ trễ thấp, chi phí xử lý nhỏ và phù hợp với ứng dụng thời gian thực.

Trong phạm vi đề tài, thư viện `socket` được sử dụng như sau:

- **Unicast:** gửi dữ liệu đến một địa chỉ IP cụ thể.
- **Broadcast trên IPv4:** bật tùy chọn Broadcast và gửi dữ liệu đến địa chỉ quảng bá của mạng con.
- **Multicast:** tạo UDP socket, tham gia nhóm và gửi/nhận dữ liệu qua địa chỉ Multicast.
- **Anycast trên IPv6:** sử dụng socket tương tự Unicast; hệ thống định tuyến lựa chọn nút có đường đi tối ưu trong nhóm cùng chia sẻ địa chỉ.

3.6 Vai trò của tường lửa trong kiểm soát lưu lượng

Kế thừa nội dung tuần 2, tường lửa trên Raspberry Pi (iptables/ip6tables) lọc gói tin theo các tiêu chí như địa chỉ MAC nguồn, địa chỉ IP nguồn/đích, cổng dịch vụ và giao thức tầng vận chuyển. Trong bài thực hành tuần 3, tường lửa được cấu hình để:

1. Chặn hoàn toàn lưu lượng Unicast không mong muốn.
2. Chỉ cho phép Broadcast, Multicast và Anycast đi qua các cổng dịch vụ và nhóm địa chỉ đã khai báo tường minh.

Đây là mô hình *default deny, explicit allow*: mặc định từ chối và chỉ mở đúng những lưu lượng cần thiết, qua đó giảm bề mặt tấn công.

4 Mô hình triển khai

4.1 Vai trò và địa chỉ IP của thiết bị

Bảng 1: Vai trò và cấu hình địa chỉ trong mô hình thực nghiệm.

Vai trò	Thiết bị	Địa chỉ
Thiết bị 1 – Sender	Windows 1	IPv4: `192.168.1.153` Broadcast: `192.168.1.255` IPv6: `fd53:1234:5678:1::80`
Thiết bị 2 – Receiver chính, có tường lửa	Raspberry Pi (WebOS)	IPv4: `192.168.1.122` Multicast IPv4: `224.1.1.1` IPv6: `fd53:1234:5678:1::10` Anycast: `fd53:1234:5678:1::100` Multicast IPv6: `ff15::100`
Thiết bị 3 – Receiver đối chứng, không tường lửa	Windows 2	IPv4: `192.168.1.193` Multicast IPv4: `224.12.1.1` IPv6: `fd53:1234:5678:1::50` Anycast: `fd53:1234:5678:1::100` Multicast IPv6: `ff15::200`

Ba thiết bị được kết nối qua cùng một LAN, sử dụng subnet `192.168.1.0/24`. Dải địa chỉ này khác với subnet của phòng LAB gốc nhằm tránh xung đột và cô lập lưu lượng thử nghiệm.

4.2 Vai trò của Windows 2

Windows 2 không áp dụng rule tường lửa và được dùng làm thiết bị đối chứng. Việc so sánh Windows 2 với Raspberry Pi khi cùng nhận gói Broadcast hoặc Multicast giúp khẳng định sự khác biệt về kết quả là do rule tường lửa, không phải do lỗi cấu hình mạng. Thiết bị được cho tham gia các nhóm Multicast thông qua chương trình nhận tương ứng.

4.3 Chương trình Python gửi và nhận

Hệ thống gồm chương trình sender chạy trên Windows 1, sử dụng Scapy hoặc socket thuần để tạo và gửi gói Unicast, Broadcast, Multicast và Anycast; các chương trình receiver chạy trên Raspberry Pi và Windows 2 để lắng nghe tại cổng hoặc nhóm địa chỉ tương ứng.

4.3.1 IPv4 Unicast

unicast_send.py

```
1  import socket
2  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
3  sock.sendto(b"Hello Unicast from Win 1", ("192.168.1.122", 5009))
```

Hình 5: Chương trình gửi gói IPv4 Unicast.

unicast_receive.py

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", 5009))
print("Listening Unicast IPv4 on 5009...")
while True:
    data, addr = sock.recvfrom(1024)
    print(f"RECIEVE: {data.decode()} từ {addr}")
```

Hình 6: Chương trình nhận gói IPv4 Unicast.

4.3.2 IPv4 Broadcast

broadcast_send.py

```python
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
sock.sendto(b"hello-broadcast", ("192.168.1.255", 5008))
```

Hình 7: Chương trình gửi gói IPv4 Broadcast.

broadcast_receive.py

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(("", 5008))

while True:
    data, addr = sock.recvfrom(1024)
    print(f"Received {data} from {addr}")
```

Hình 8: Chương trình nhận gói IPv4 Broadcast.

4.3.3 IPv4 Multicast

multi_send (1).py

```python
import socket

MCAST_PORT = 5007
SENDER_IP = "192.168.1.144"
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_IF, socket.inet_aton(SENDER_IP))

sock.sendto(b"Message Group 224.1.1.1 (Rasp)", ("224.1.1.1", MCAST_PORT))

sock.sendto(b"Message Group 224.12.1.1 (Win2)", ("224.12.1.1", MCAST_PORT))
```

Hình 9: Chương trình gửi gói IPv4 Multicast.

multi_receive.py

```python
import socket, struct

MCAST_GRP = '224.1.1.1'
MCAST_PORT = 5007

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(('0.0.0.0', MCAST_PORT))
mreq = struct.pack("4sl", socket.inet_aton(MCAST_GRP), socket.INADDR_ANY)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)

print(f"Raspberry Pi listening Multicast group {MCAST_GRP}...")
while True:
    print(sock.recv(1024).decode())
```

Hình 10: Chương trình nhận IPv4 Multicast trên Raspberry Pi.

multi_receive2.py

```python
import socket, struct

MCAST_GRP = '224.12.1.1'
MCAST_PORT = 5007
LOCAL_IP = "192.168.1.193"

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(('', MCAST_PORT))
mreq = struct.pack("4s4s", socket.inet_aton(MCAST_GRP), socket.inet_aton(LOCAL_IP))
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)

print(f"Listening on {MCAST_GRP}...")
while True:
    print(sock.recv(1024).decode())
```

Hình 11: Chương trình nhận IPv4 Multicast trên Windows 2.

4.3.4 IPv6 Unicast

unicast_send.py

```python
import socket

DEST_IP = "fd53:1234:5678:1::10"
PORT = 5009

sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)

sock.sendto(
    b"Hello IPv6 Unicast from Win 1",
    (DEST_IP, PORT)
)

print("Message sent.")
```

Hình 12: Chương trình gửi gói IPv6 Unicast.

anycast_receiver.py

```python
import socket

PORT = 5009

sock = socket.socket(socket.AF_INET6,
                     socket.SOCK_DGRAM)

sock.bind(("::", PORT))

print("Anycast Server Listening...")

while True:
    data, addr = sock.recvfrom(1024)

    print("Receive:", data.decode())
    print("From:", addr[0])
```

Hình 13: Chương trình nhận gói IPv6 Unicast.

4.3.5 IPv6 Multicast

multi_send.py

```python
import socket

MCAST_PORT = 5007

# Nhóm multicast IPv6
GROUP1 = "ff15::100"
GROUP2 = "ff15::200"

# Tạo socket IPv6 UDP
sock = socket.socket(socket.AF_INET6,
                     socket.SOCK_DGRAM,
                     socket.IPPROTO_UDP)

# Hop Limit (TTL của IPv6)
sock.setsockopt(socket.IPPROTO_IPV6,
                socket.IPV6_MULTICAST_HOPS,
                2)

# Gửi tới Group 1
sock.sendto(
    b"Message Group ff15::100 (Rasp)",
    (GROUP1, MCAST_PORT)
)

# Gửi tới Group 2
sock.sendto(
    b"Message Group ff15::200 (Win2)",
    (GROUP2, MCAST_PORT)
)

print("Send OK")
```

Hình 14: Chương trình gửi gói IPv6 Multicast.

IPv6_Multicast_Receiver.py

```python
import socket, struct

GROUP = "ff15::200"
PORT = 5007
IFACE_INDEX = 19   # đổi đúng tên adapter

sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(('', PORT))
mreq = socket.inet_pton(socket.AF_INET6, GROUP) + struct.pack('@I', IFACE_INDEX)
sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_JOIN_GROUP, mreq)

print(f"Windows#2 dang nghe {GROUP}...")
while True:
    print(sock.recv(10240).decode())
```

Hình 15: Chương trình nhận gói IPv6 Multicast.

4.3.6 IPv6 Anycast

anycast_send.py

```python
import socket

ANYCAST_IP = "fd53:1234:5678:1::10"

PORT = 5009

sock = socket.socket(socket.AF_INET6,
                     socket.SOCK_DGRAM)

sock.sendto(
    b"Hello Anycast",
    (ANYCAST_IP, PORT)
)

print("Send OK")
```

Hình 16: Chương trình gửi gói IPv6 Anycast.

anycast_receiver.py

```python
import socket

PORT = 5009

sock = socket.socket(socket.AF_INET6,
                     socket.SOCK_DGRAM)

sock.bind(("::", PORT))

print("Anycast Server Listening...")

while True:
    data, addr = sock.recvfrom(1024)

    print("Receive:", data.decode())
    print("From:", addr[0])
```

Hình 17: Chương trình nhận gói IPv6 Anycast.

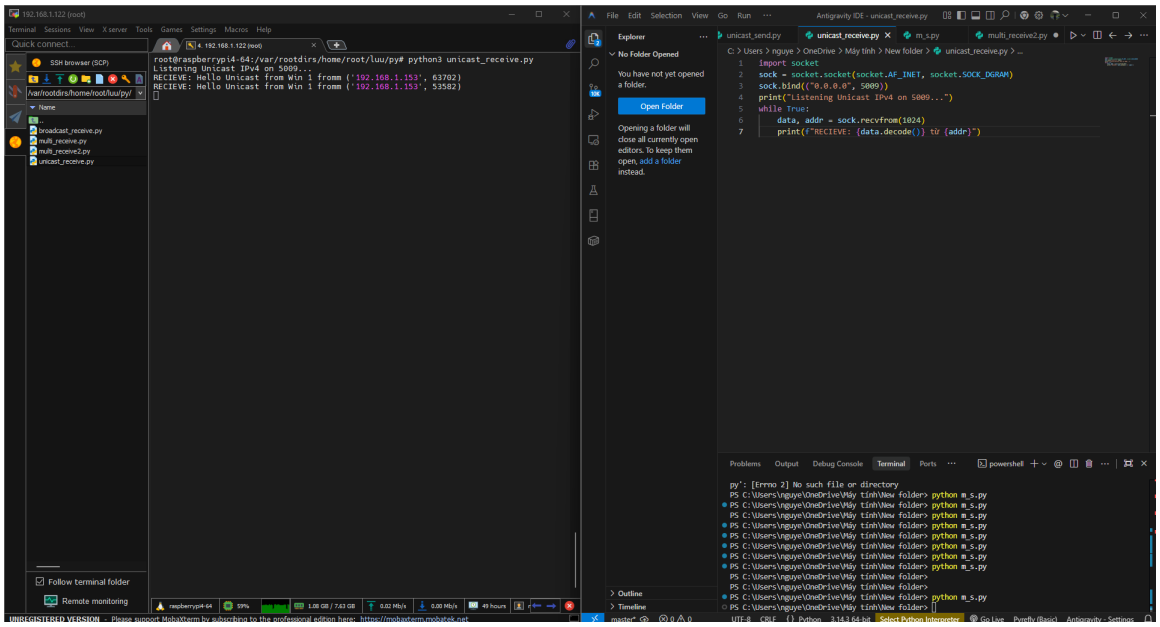
5 Triển khai và đánh giá kết quả

Nhóm thực hiện đối chứng theo phương pháp của tuần 2 (kịch bản kiểm tra cơ chế chặn của tường lửa). Mỗi loại lưu lượng được gửi ở hai trạng thái: **trước khi áp dụng rule** (baseline) và **sau khi áp dụng rule**. Cách làm này giúp tách bạch nguyên nhân thành công hoặc thất bại là do rule lọc, không phải do lỗi mạng.

5.1 Trạng thái trước khi áp dụng rule tường lửa

5.1.1 IPv4 Unicast

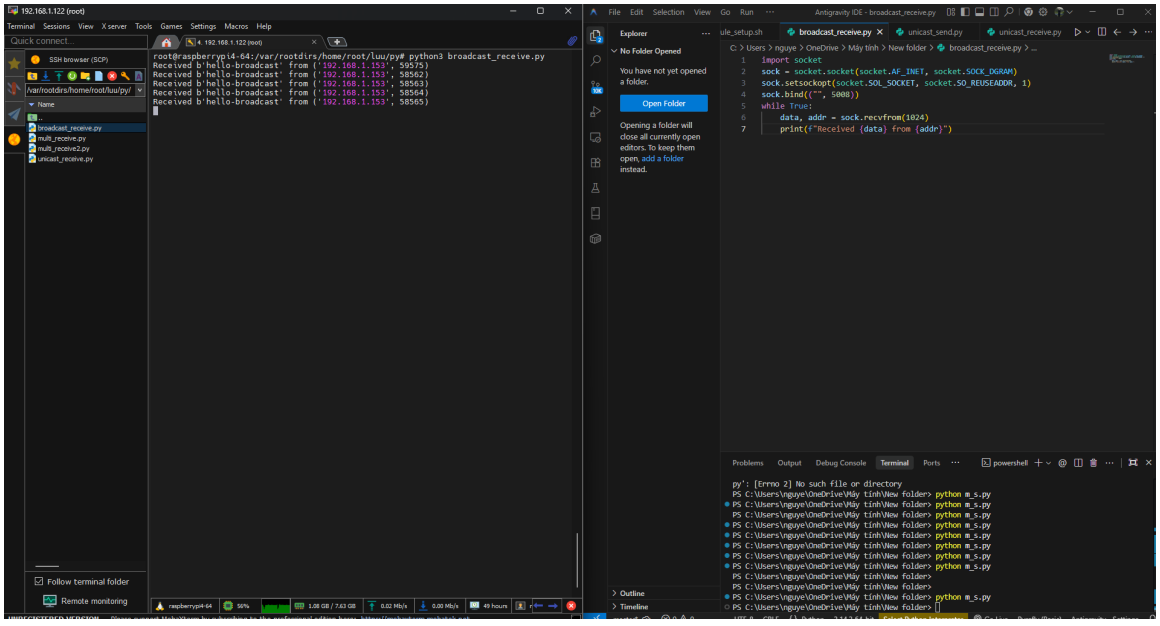
Raspberry Pi nhận thành công gói IPv4 Unicast gửi từ Windows 1. Đây là kết quả nền để so sánh với trạng thái sau khi áp dụng rule.



Hình 18: Raspberry Pi nhận thành công lưu lượng IPv4 Unicast ở trạng thái baseline.

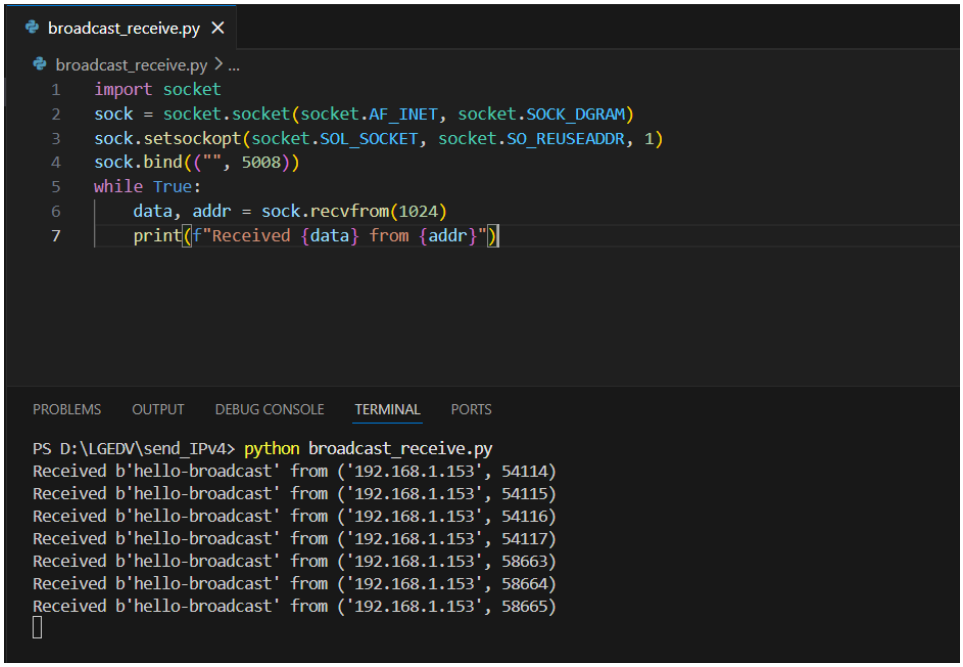
5.1.2 IPv4 Broadcast

Cả Raspberry Pi và Windows 2 đều nhận được gói IPv4 Broadcast khi chưa có rule tường lửa. Kết quả phù hợp với bản chất một–đến–tất cả của Broadcast: mọi thiết bị trong cùng miền quảng bá đều nhận gói, không phụ thuộc việc đăng ký nhóm.



Hình 19: Raspberry Pi nhận gói IPv4 Broadcast trước khi áp dụng rule.

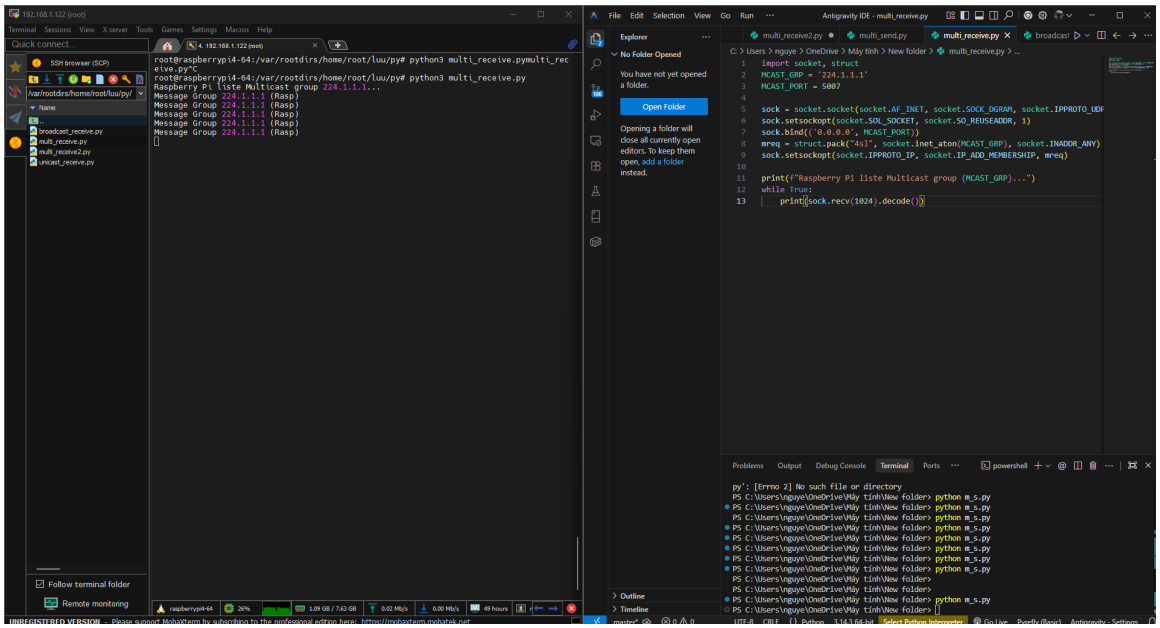
Windows 2 cũng nhận thành công các gói Broadcast từ máy Sender.



Hình 20: Windows 2 nhận thành công các gói IPv4 Broadcast.

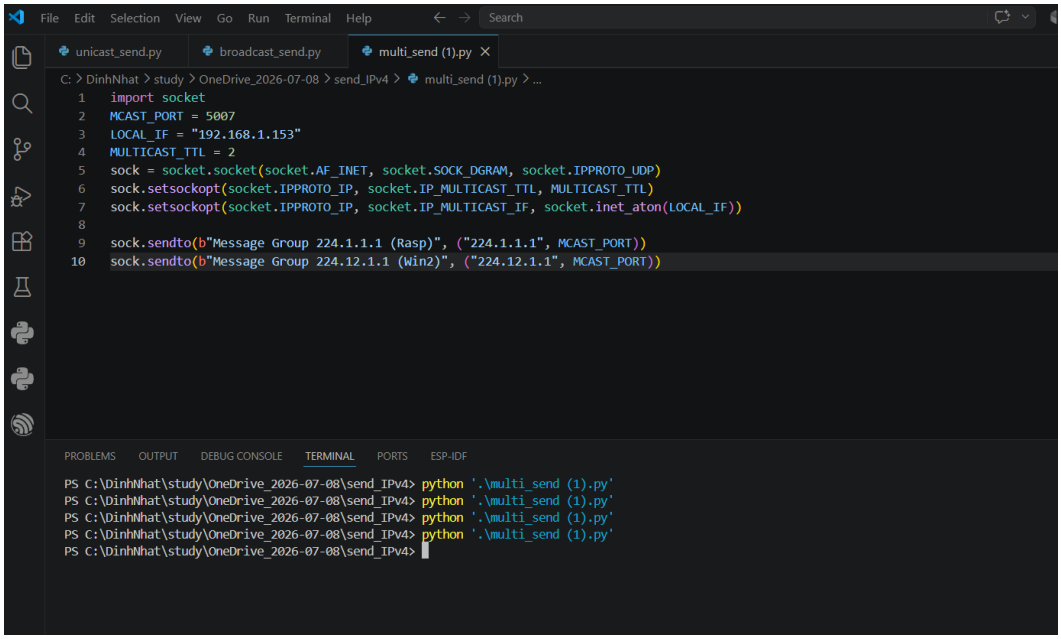
5.1.3 IPv4 Multicast

Raspberry Pi, sau khi gia nhập nhóm `224.1.1.1`, nhận được gói Multicast tương ứng. Khi chuyển sang nhóm `224.12.1.1`, thiết bị bắt đầu nhận được gói gửi tới nhóm mới từ Windows 1. Kết quả xác nhận cơ chế IGMP group membership hoạt động đúng: thiết bị chỉ nhận gói của nhóm mà nó đã tham gia.



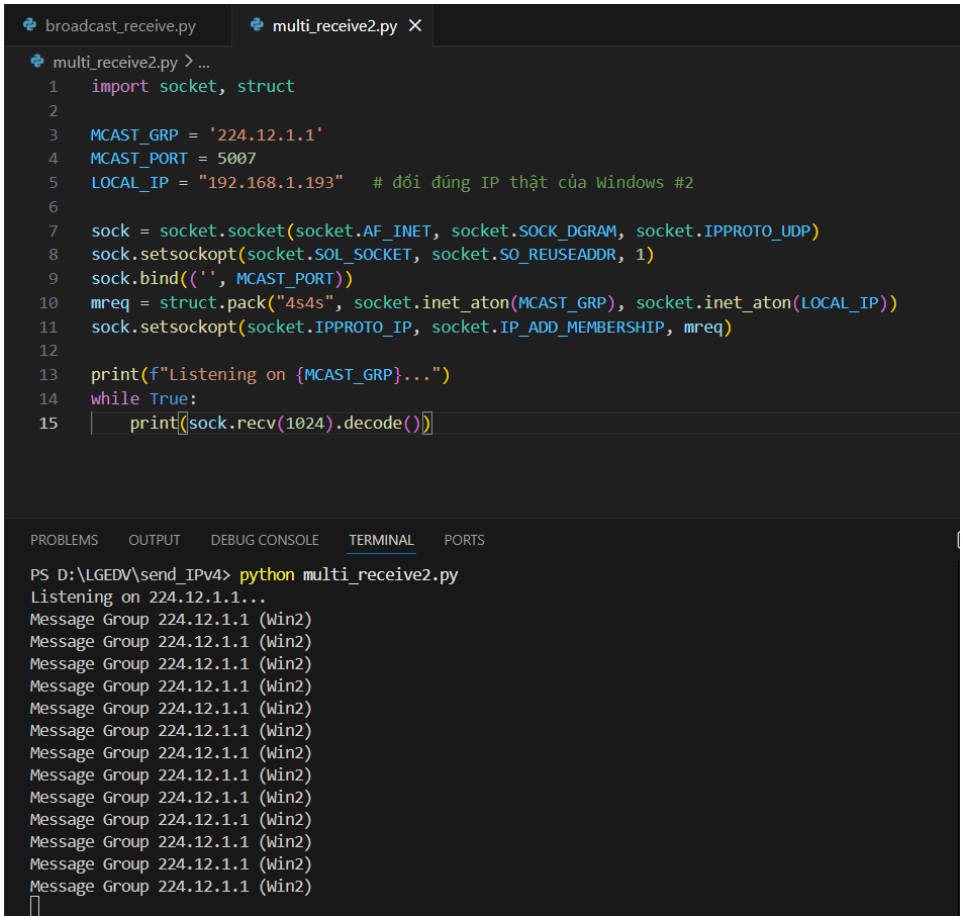
Hình 21: Raspberry Pi nhận lưu lượng IPv4 Multicast theo nhóm đã tham gia.

Máy Sender gửi IPv4 Multicast đến hai nhóm: một nhóm dành cho Raspberry Pi và một nhóm dành cho Windows 2.



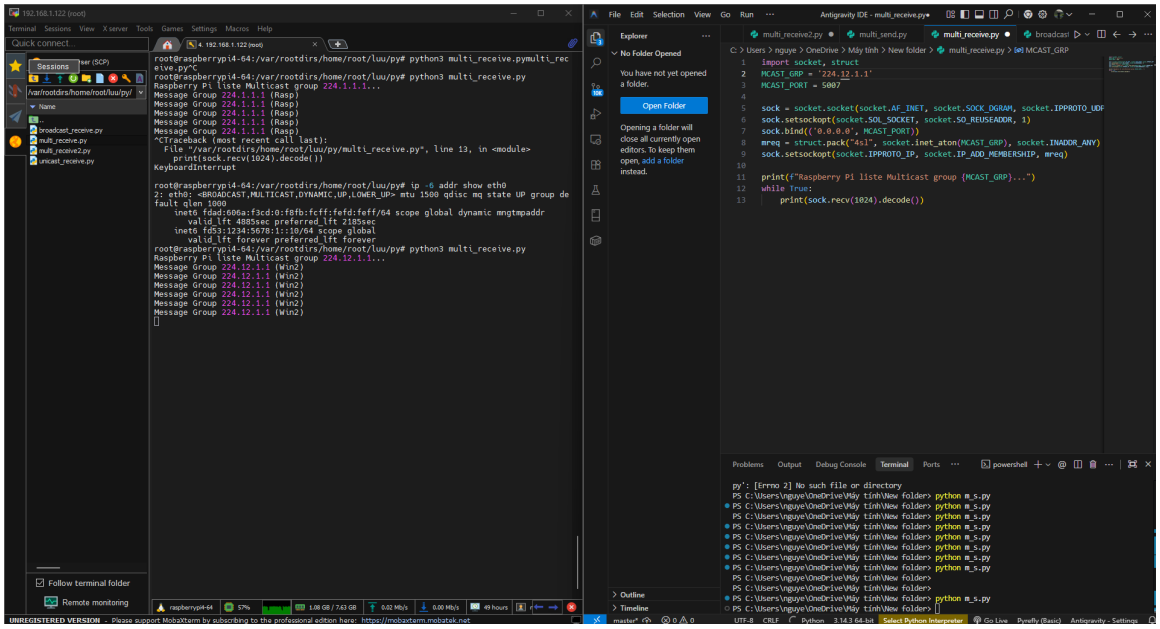
Hình 22: Windows 1 gửi IPv4 Multicast đến hai nhóm thử nghiệm.

Windows 2 tham gia nhóm `224.12.1.1` và nhận thành công các gói do Windows 1 gửi.

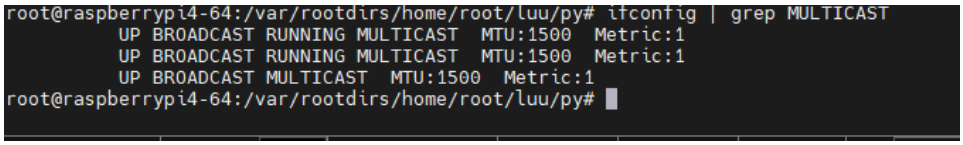


Hình 23: Windows 2 nhận IPv4 Multicast của nhóm `224.12.1.1`.

Sau khi Raspberry Pi đổi sang nhóm `224.12.1.1`, thiết bị nhận được lưu lượng tương ứng.



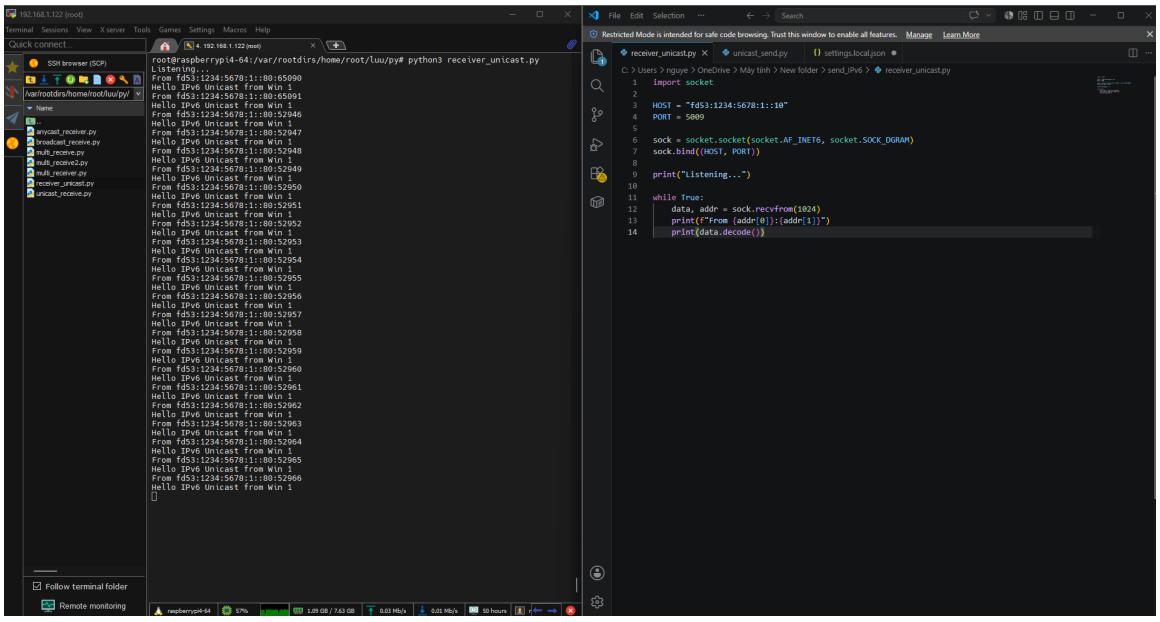
Hình 24: Raspberry Pi nhận lưu lượng sau khi chuyển nhóm IPv4 Multicast.



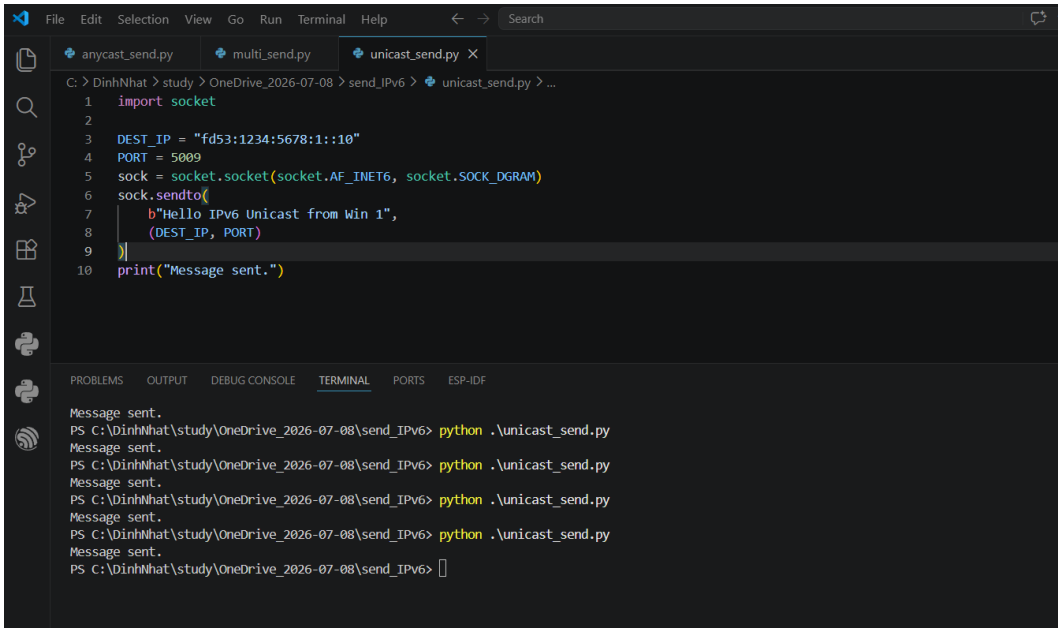
Hình 25: Thông tin IGMP xác nhận Raspberry Pi đã gia nhập nhóm Multicast.

5.1.4 IPv6 Unicast

Raspberry Pi nhận được gói IPv6 Unicast từ Windows 1 với địa chỉ nguồn `fd53:1234:5678:1::80`.



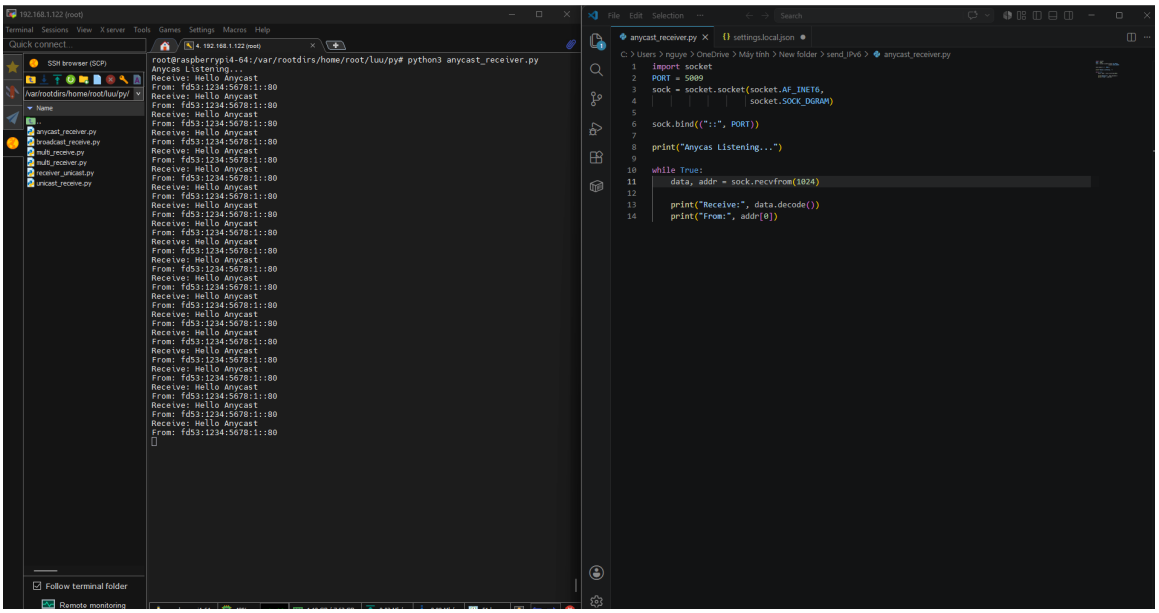
Hình 26: Raspberry Pi nhận gói IPv6 Unicast ở trạng thái baseline.



Hình 27: Windows 1 gửi lưu lượng IPv6 Unicast.

5.1.5 IPv6 Anycast

Raspberry Pi nhận thành công gói IPv6 Anycast gửi từ Windows 1 thông qua địa chỉ dùng chung `fd53:1234:5678:1::100`.



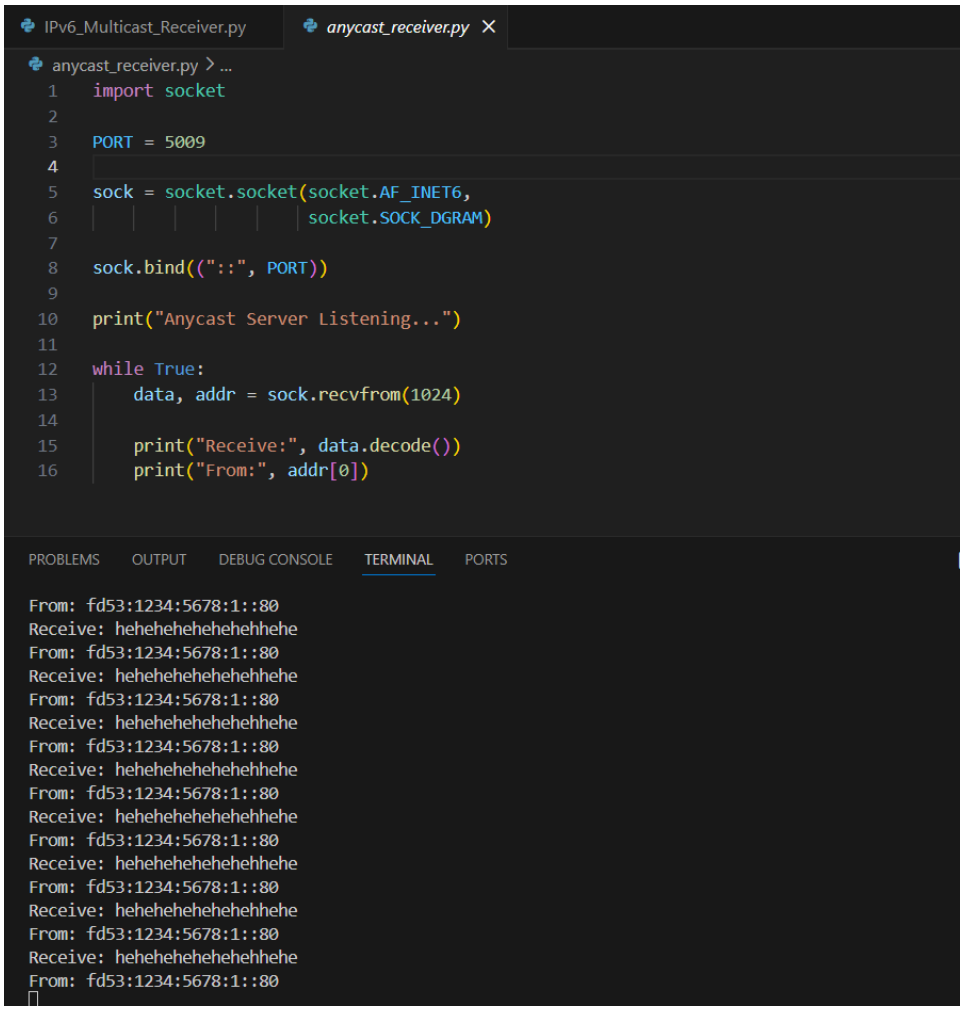
Hình 28: Raspberry Pi nhận thành công gói IPv6 Anycast.

Địa chỉ Anycast được cấu hình trên các thiết bị nhận bằng lệnh:

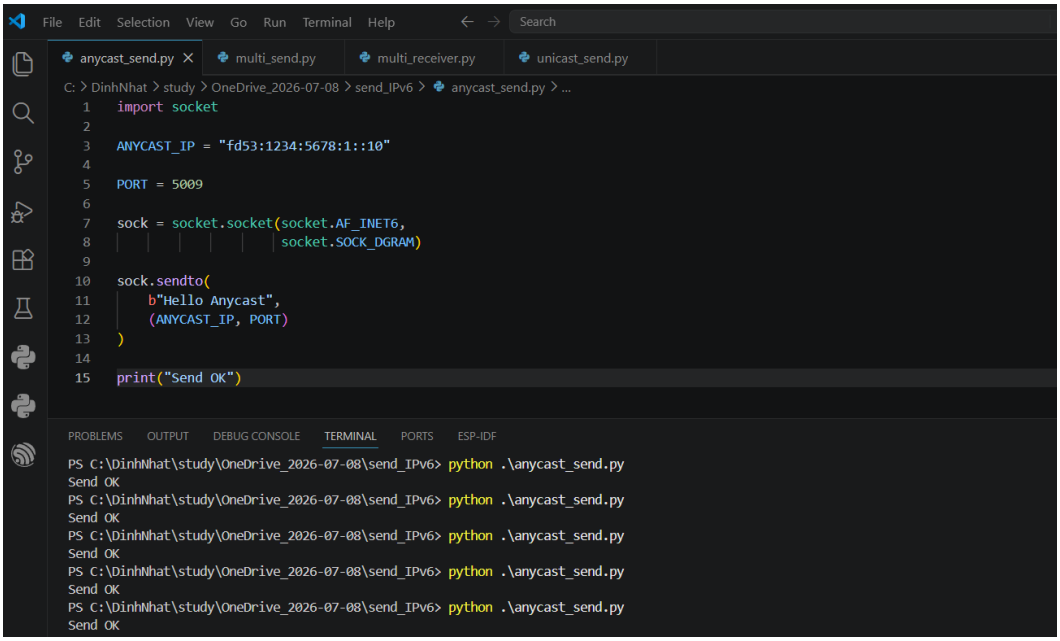
```
ip -6 addr add fd53:1234:5678:1::100/64 dev eth0 nodad
```

```
eth0: <BROADCAST,MULTICAST,DYNAMIC,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
 link/ether fa:fb:fc:fd:fe:ff brd ff:ff:ff:ff:ff:ff permaddr d8:3a:dd:e5:32:07
 inet 192.168.1.122/24 brd 192.168.1.255 scope global eth0
    valid_lft forever preferred_lft forever
 inet6 fd53:1234:5678:1::100/64 scope global nodad
    valid_lft forever preferred_lft forever
 inet6 fd53:1234:5678:1::10/64 scope global
    valid_lft forever preferred_lft forever
```

Hình 29: Cấu hình địa chỉ IPv6 Anycast trên Raspberry Pi.



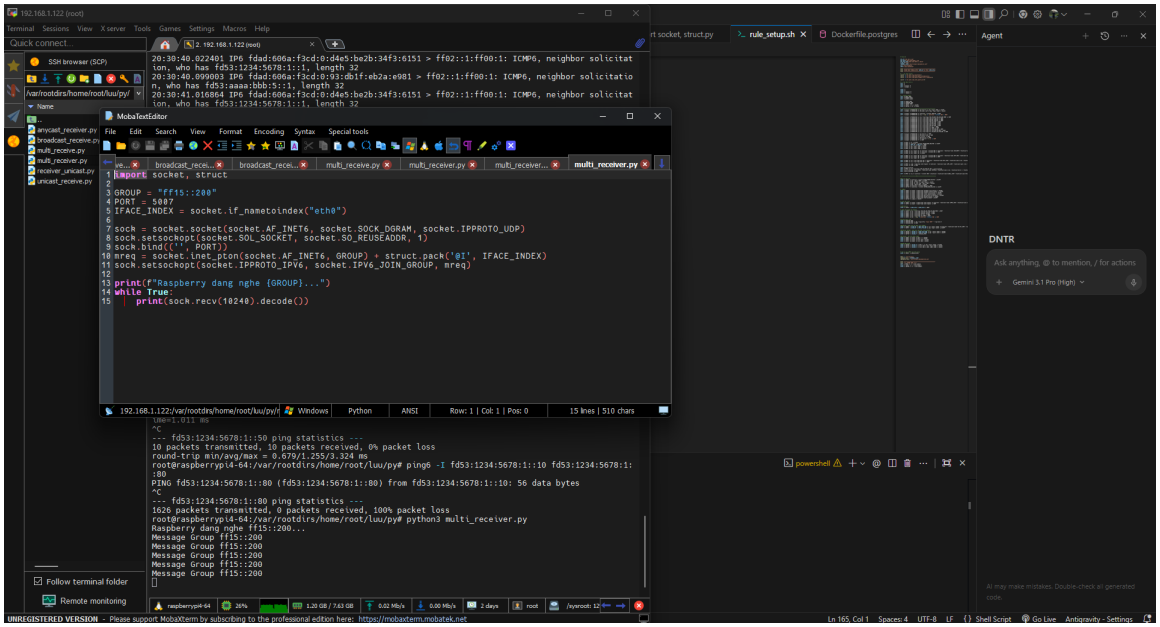
Hình 30: Windows 2 nhận lưu lượng tại địa chỉ IPv6 Anycast dùng chung.



Hình 31: Windows 1 gửi lưu lượng IPv6 Anycast.

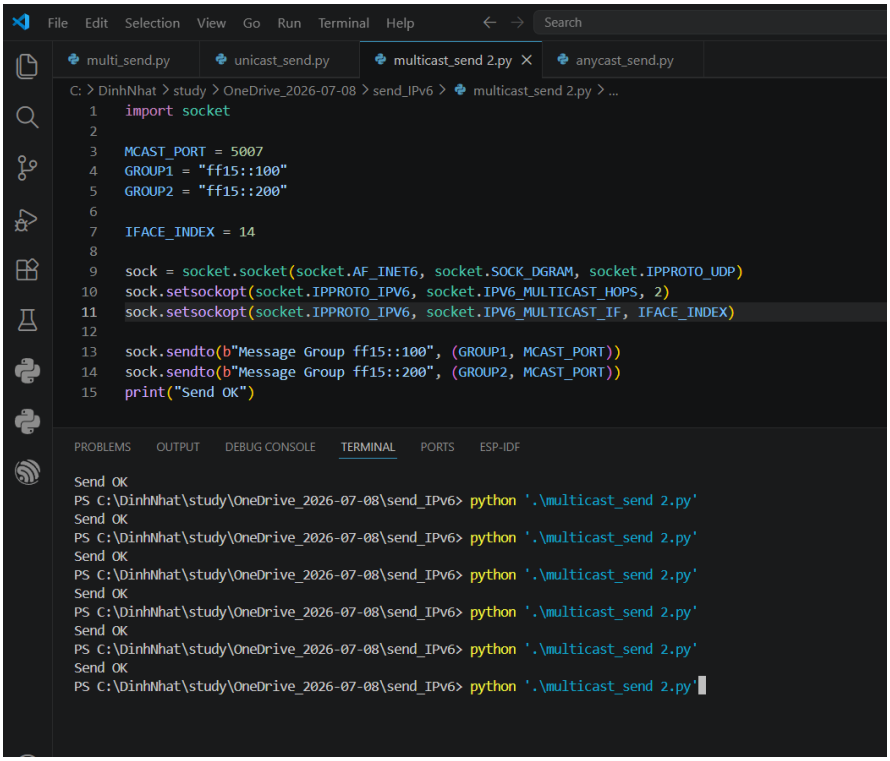
5.1.6 IPv6 Multicast

Raspberry Pi gia nhập nhóm `ff15::200`. Khi Windows 1 gửi gói đến nhóm `ff15::100`, Raspberry Pi không nhận được do khác nhóm; thiết bị chỉ nhận gói gửi đến đúng nhóm đã tham gia. Kết quả tiếp tục xác nhận nguyên lý lọc theo group membership, độc lập với tường lửa.



Hình 32: Kết quả nhận IPv6 Multicast trên Raspberry Pi theo đúng nhóm.

Windows 1 gửi Multicast đến hai nhóm, trong đó mỗi receiver tham gia một nhóm riêng.



Hình 33: Windows 1 gửi IPv6 Multicast đến hai nhóm thử nghiệm.



Hình 34: Gửi lưu lượng đến nhóm IPv6 Multicast dành cho Raspberry Pi.

Windows 2 nhận thành công các gói IPv6 Multicast từ nhóm `ff15::200`.

broadcast_receive.py receiver_unicast.py anycast_receiver.py IPv6_Multicast_Receiver.py

D: > LGEDV > send_IPv6 > IPv6_Multicast_Receiver.py > ...

```python
import socket, struct

GROUP = "ff15::200"
PORT = 5007
IFACE_INDEX = 19   # đổi đúng tên adapter

sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(('', PORT))
mreq = socket.inet_pton(socket.AF_INET6, GROUP) + struct.pack('@I', IFACE_INDEX)
sock.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_JOIN_GROUP, mreq)

print(f"Windows#2 dang nghe {GROUP}...")
while True:
    print(sock.recv(10240).decode())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\LGEDV\send_IPv4> cd D:\LGEDV\send_IPv6
PS D:\LGEDV\send_IPv6> python IPv6_Multicast_Receiver.py
Windows#2 dang nghe ff15::200...
Message Group ff15::200
Message Group ff15::200
Message Group ff15::200
Message Group ff15::200
Message Group ff15::200
Message Group ff15::200
Message Group ff15::200
```

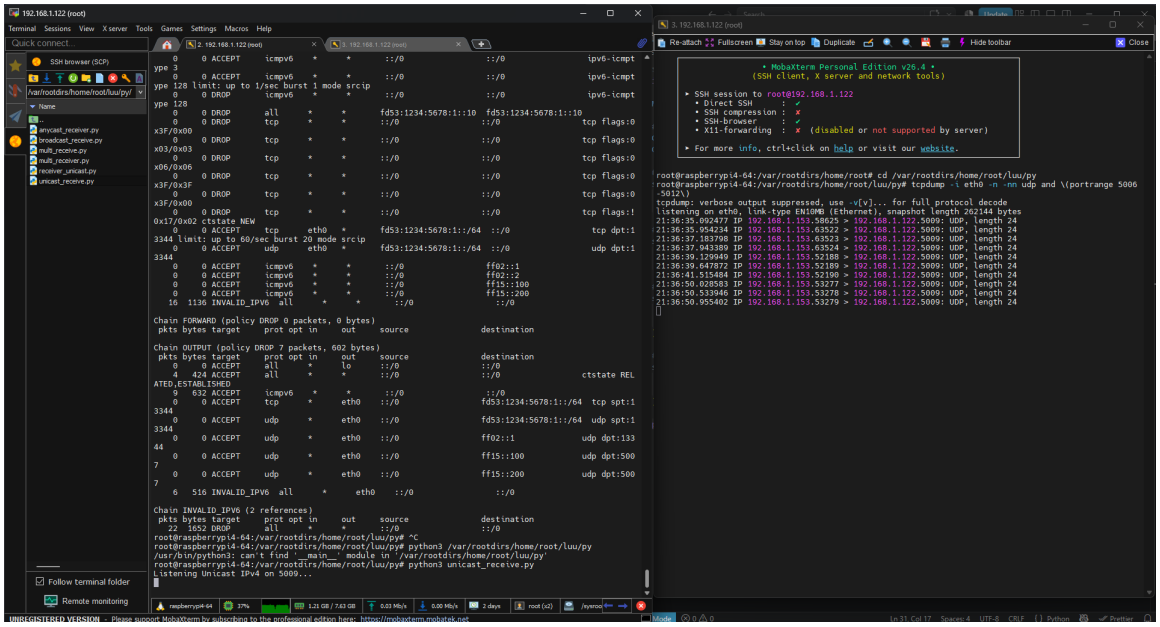
Hình 35: Windows 2 nhận IPv6 Multicast từ nhóm `ff15::200`.

5.2 Trạng thái sau khi áp dụng rule tường lửa

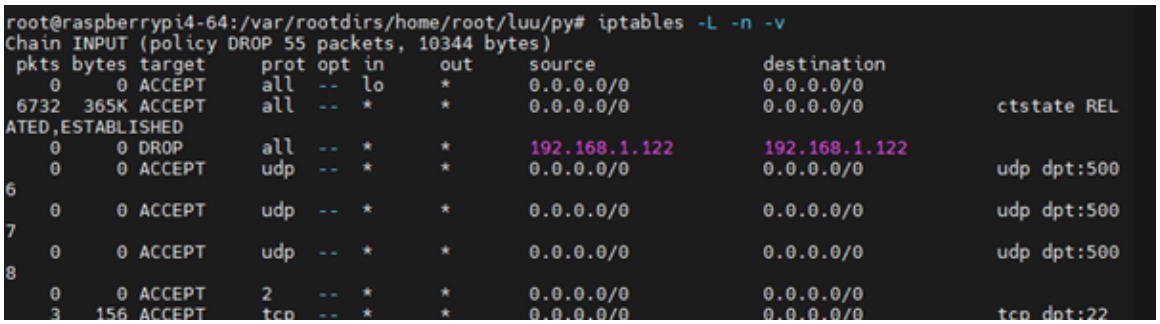
Ở giai đoạn này, Raspberry Pi được bổ sung các rule để chỉ cho phép lưu lượng hợp lệ đi qua theo đúng cổng dịch vụ và nhóm địa chỉ đã khai báo, mô phỏng một chính sách kiểm soát truy cập thực tế.

5.2.1 IPv4 Unicast – BỊ CHẶN

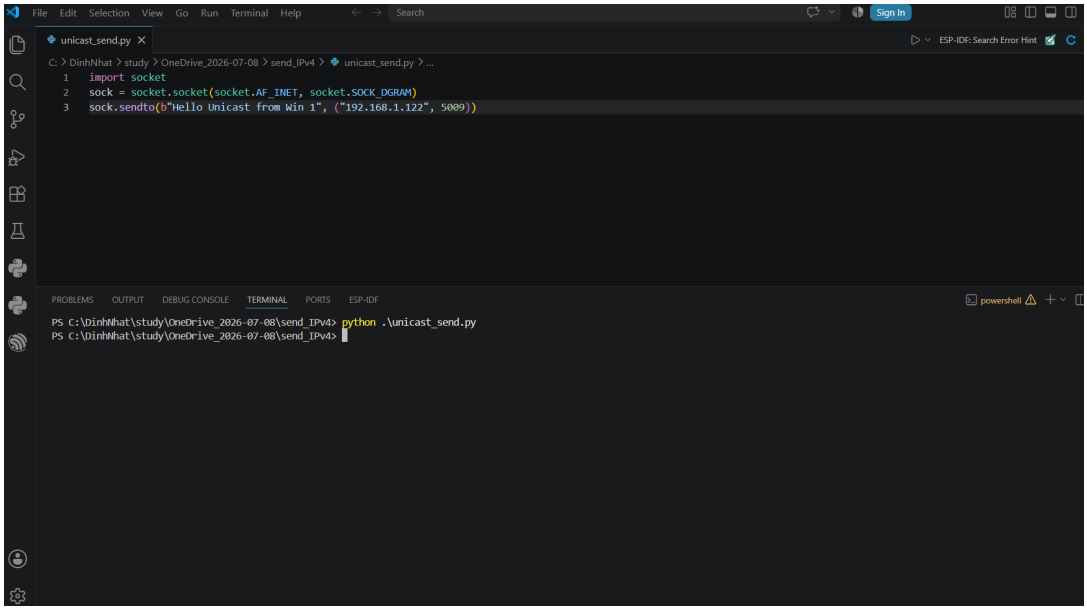
Windows 1 gửi 10 gói Unicast đến Raspberry Pi nhưng không nhận được phản hồi do tường lửa drop. Bộ đếm của rule ghi nhận đúng 10 gói bị loại bỏ, xác nhận toàn bộ lưu lượng Unicast không nằm trong whitelist đều bị chặn.



Hình 36: IPv4 Unicast không đến được chương trình nhận sau khi áp dụng rule.



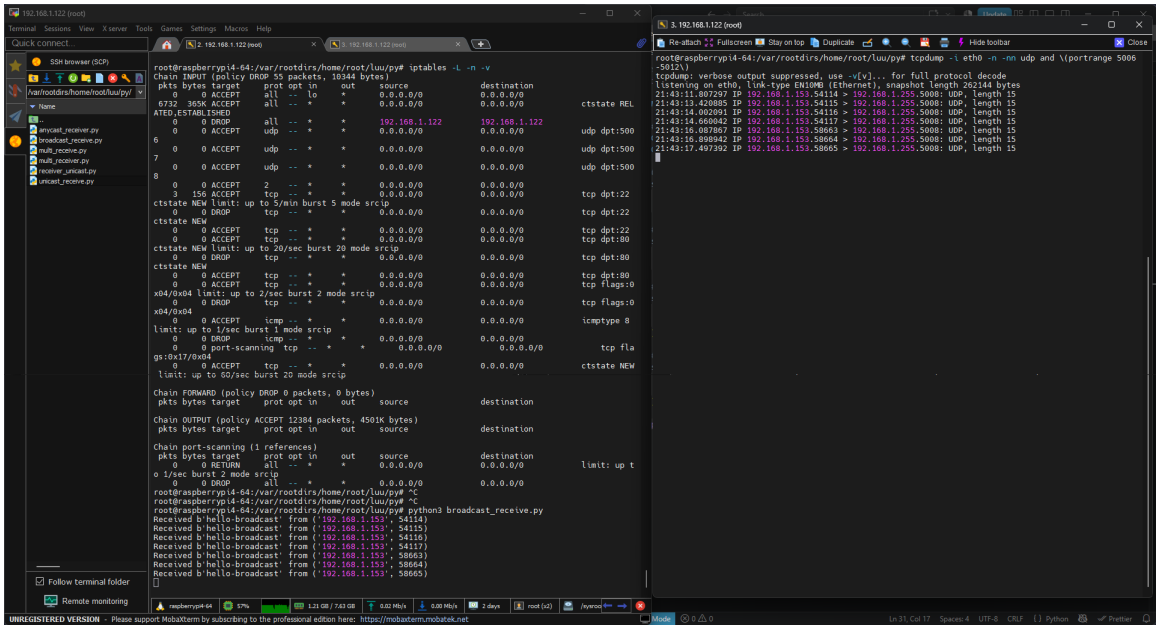
Hình 37: Bộ đếm iptables ghi nhận 10 gói IPv4 Unicast bị drop.



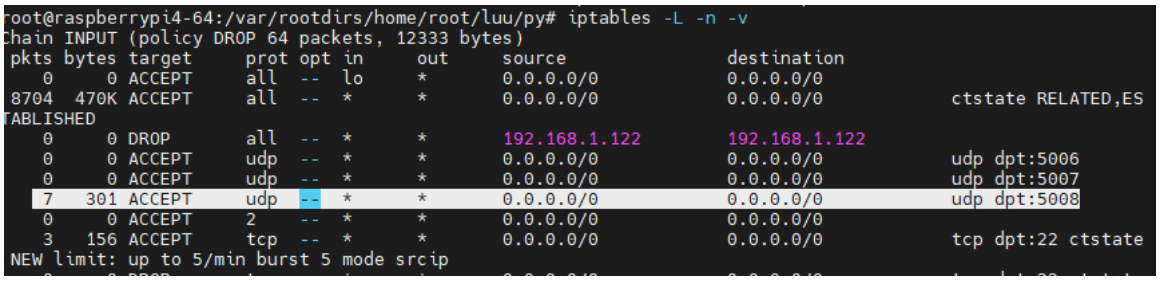
Hình 38: Windows 1 gửi IPv4 Unicast đến `192.168.1.122`.

5.2.2 IPv4 Broadcast – ĐƯỢC CHO PHÉP

Windows 1 gửi Broadcast đến `192.168.1.255` trên cổng `5008`. Cổng này nằm trong whitelist nên gói tin được chuyển tiếp và xử lý bình thường.



Hình 39: Raspberry Pi nhận IPv4 Broadcast qua cổng được cho phép.



Hình 40: Rule iptables cho phép lưu lượng Broadcast qua cổng UDP 5008.

Windows 2 tiếp tục nhận thành công các gói Broadcast từ Windows 1 (`192.168.1.153`).

broadcast_receive.py

```python
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(("", 5008))
while True:
    data, addr = sock.recvfrom(1024)
    print(f"Received {data} from {addr}")
```

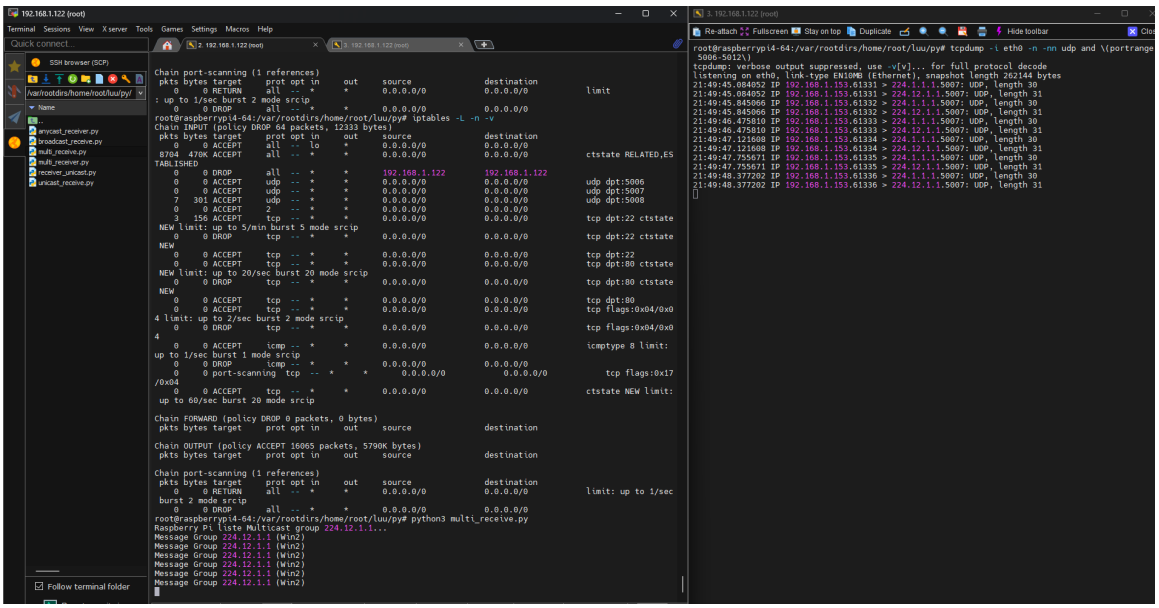
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\LGEDV\send_IPv4> python broadcast_receive.py
Received b'hello-broadcast' from ('192.168.1.153', 54114)
Received b'hello-broadcast' from ('192.168.1.153', 54115)
Received b'hello-broadcast' from ('192.168.1.153', 54116)
Received b'hello-broadcast' from ('192.168.1.153', 54117)
Received b'hello-broadcast' from ('192.168.1.153', 58663)
Received b'hello-broadcast' from ('192.168.1.153', 58664)
Received b'hello-broadcast' from ('192.168.1.153', 58665)
```

Hình 41: Thiết bị đối chứng Windows 2 nhận IPv4 Broadcast.

5.2.3 IPv4 Multicast – ĐƯỢC CHO PHÉP theo nhóm

Raspberry Pi đang tham gia nhóm `224.12.1.1` và nhận được gói Multicast từ Windows 1 trên cổng `5007`. Khi Windows 1 gửi đến `224.1.1.1` trong khi Raspberry Pi vẫn ở nhóm `224.12.1.1`, thiết bị không nhận được gói. Như vậy, group membership vẫn là điều kiện tiên quyết, độc lập với rule tường lửa theo cổng.

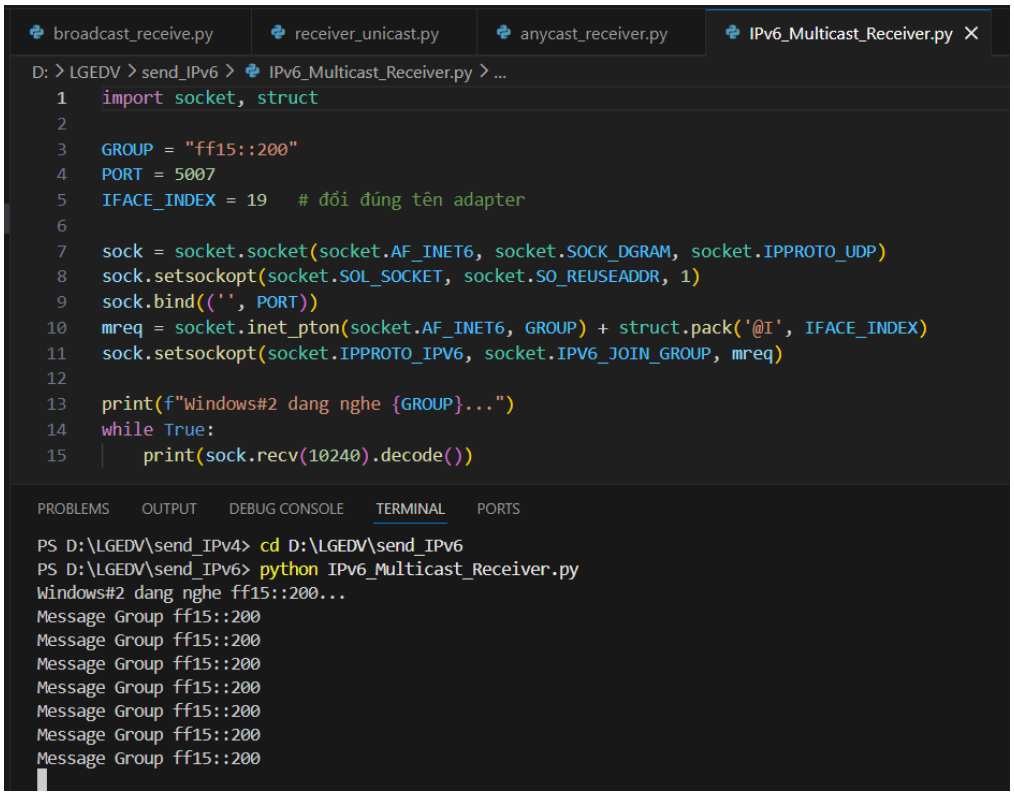


Hình 42: IPv4 Multicast được nhận khi đúng nhóm và đúng cổng dịch vụ.

```
root@raspberrypi4-64:/var/rootdirs/home/root/luu/py# iptables -L -n -v
Chain INPUT (policy DROP 73 packets, 14337 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
10063  542K ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate RELATED,ES
TABLISHED
    0     0 DROP       all  --  *      *       192.168.1.122        192.168.1.122
    0     0 ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:5006
    6   354 ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:5007
    7   301 ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:5008
    0     0 ACCEPT     2    --  *      *       0.0.0.0/0            0.0.0.0/0
    3   156 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22 ctstate
```

Hình 43: Rule iptables cho phép IPv4 Multicast qua cổng UDP 5007.

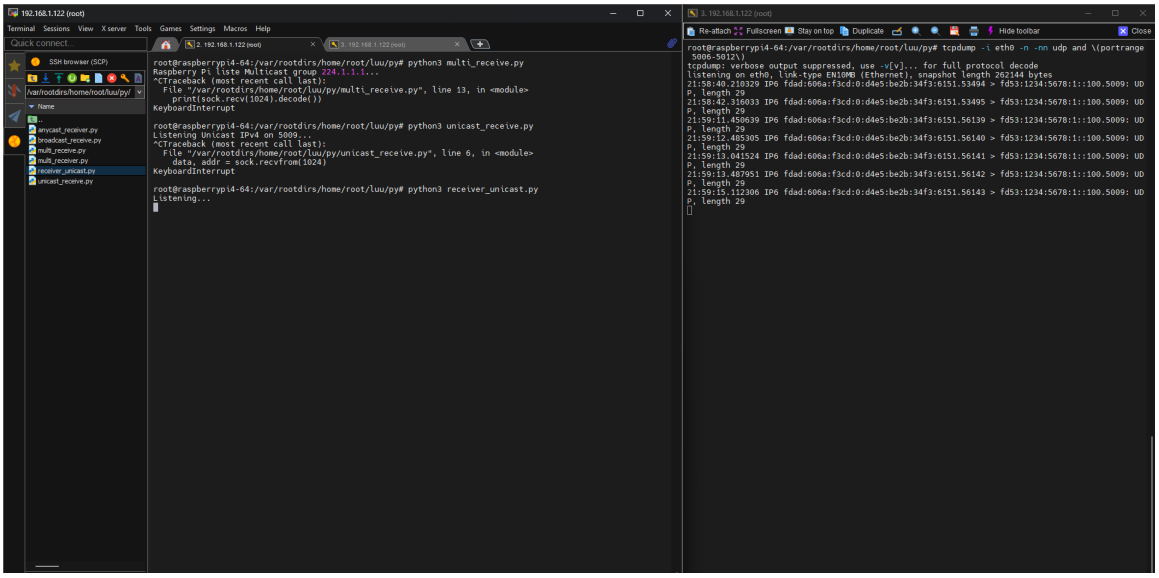
Thiết bị đối chứng Windows 2 vẫn nhận thành công lưu lượng IPv6 Multicast từ nhóm `ff15::200`; kết quả này được giữ lại để đối chiếu với Raspberry Pi có áp dụng tường lửa.



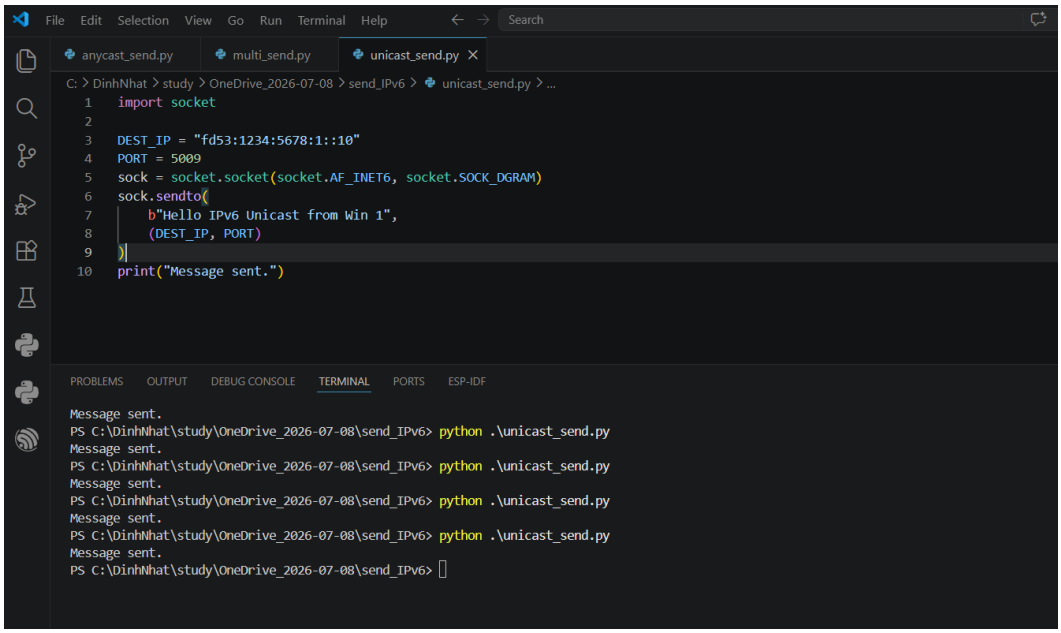
Hình 44: Windows 2 nhận IPv6 Multicast từ nhóm `ff15::200` trong phép thử đối chứng.

5.2.4 IPv6 Unicast – BỊ CHẶN

Tương tự IPv4, gói IPv6 Unicast bị tường lửa drop nên Raspberry Pi không nhận được lưu lượng từ Windows 1.



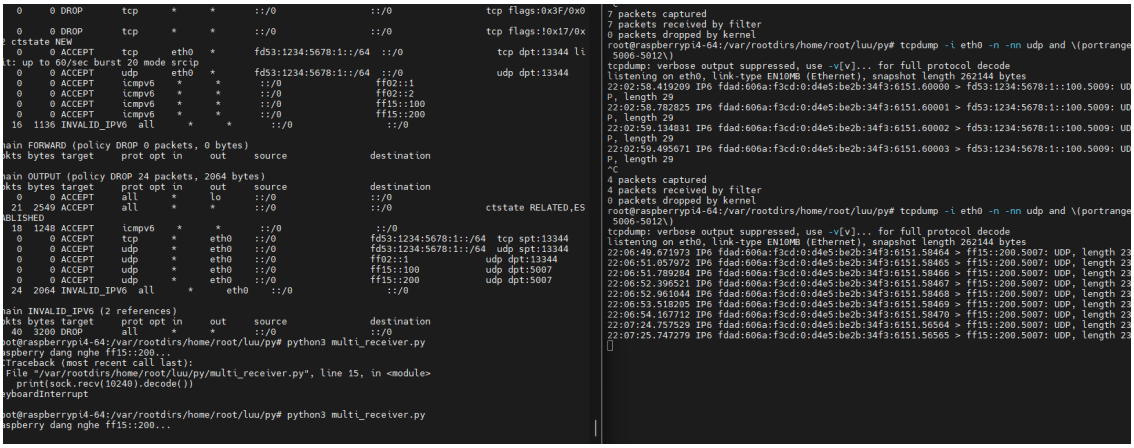
Hình 45: IPv6 Unicast bị chặn sau khi áp dụng rule tường lửa.



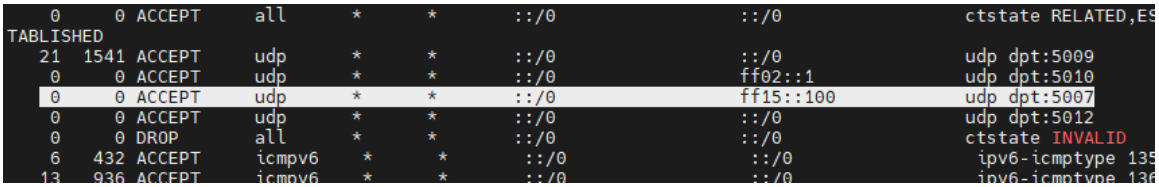
Hình 46: Phía Windows 1 vẫn thực hiện gửi IPv6 Unicast.

5.2.5 IPv6 Multicast – chỉ nhóm được khai báo mới đi qua

Rule tường lửa chỉ cho phép nhóm `ff15::100`. Khi Raspberry Pi tham gia nhóm `ff15::200`, lưu lượng của nhóm này bị chính sách mặc định drop. Kết quả cho thấy để IPv6 Multicast hoạt động sau khi có tường lửa, nhóm mà thiết bị tham gia phải khớp với nhóm được whitelist; nếu lệch nhóm, gói bị chặn dù kết nối mạng cơ bản vẫn bình thường.

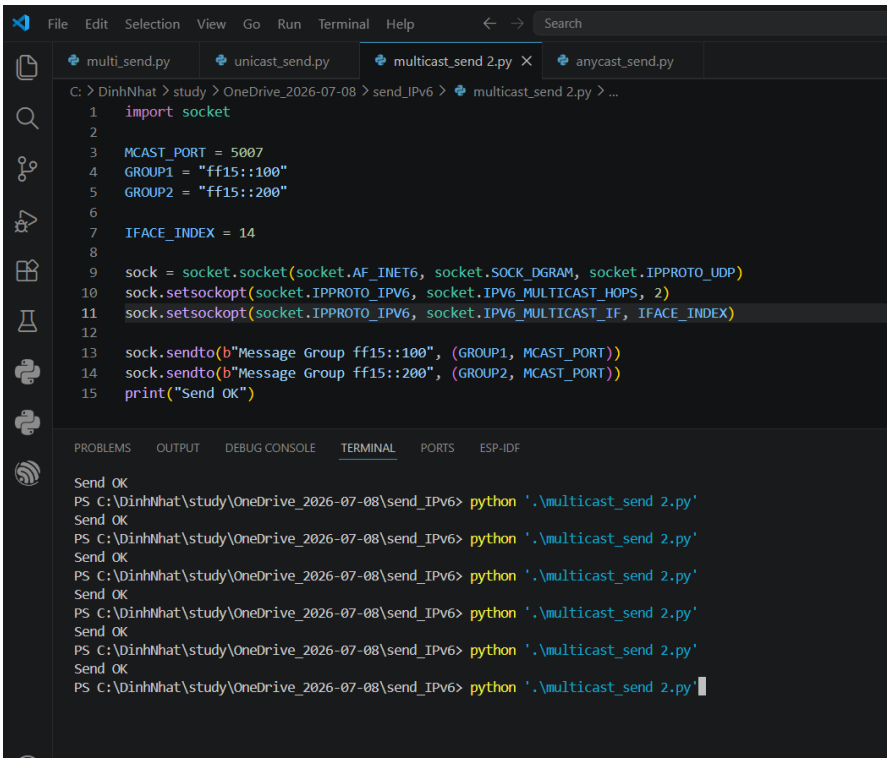


Hình 47: Lưu lượng IPv6 Multicast không khớp nhóm whitelist bị chặn.



Hình 48: Rule ip6tables chỉ cho phép nhóm `ff15::100`.

Windows 1 vẫn gửi Multicast đến hai nhóm để đối chứng.

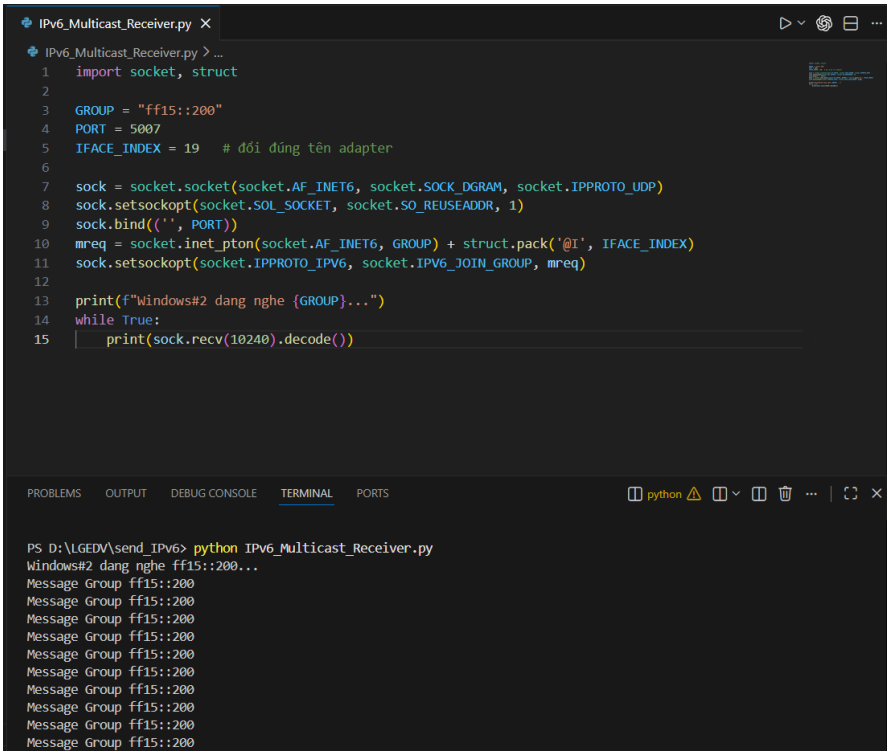


Hình 49: Chương trình gửi IPv6 Multicast đến hai nhóm.



Hình 50: Lưu lượng gửi đến từng nhóm IPv6 Multicast.

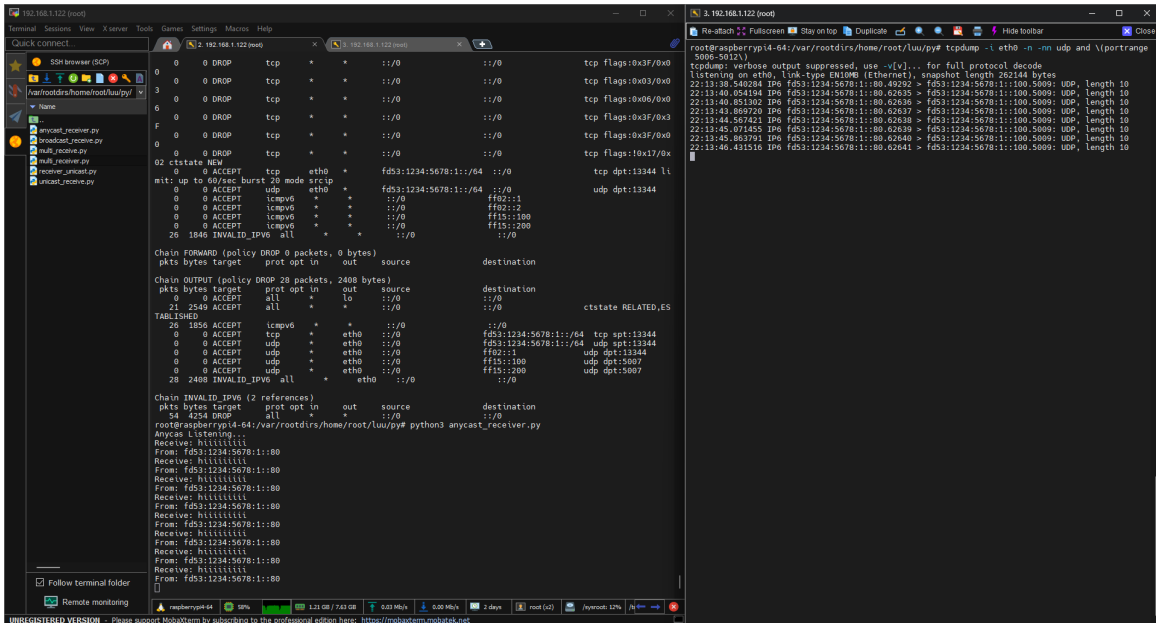
Trong khi đó, Windows 2 không áp dụng tường lửa và vẫn nhận thành công gói của nhóm `ff15::200`.



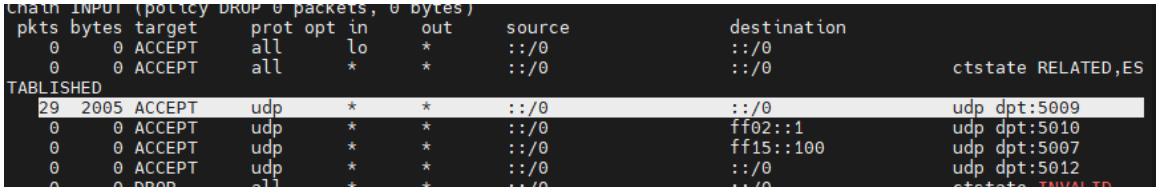
Hình 51: Windows 2 nhận thành công IPv6 Multicast từ nhóm `ff15::200`.

5.2.6 IPv6 Anycast – ĐƯỢC CHO PHÉP

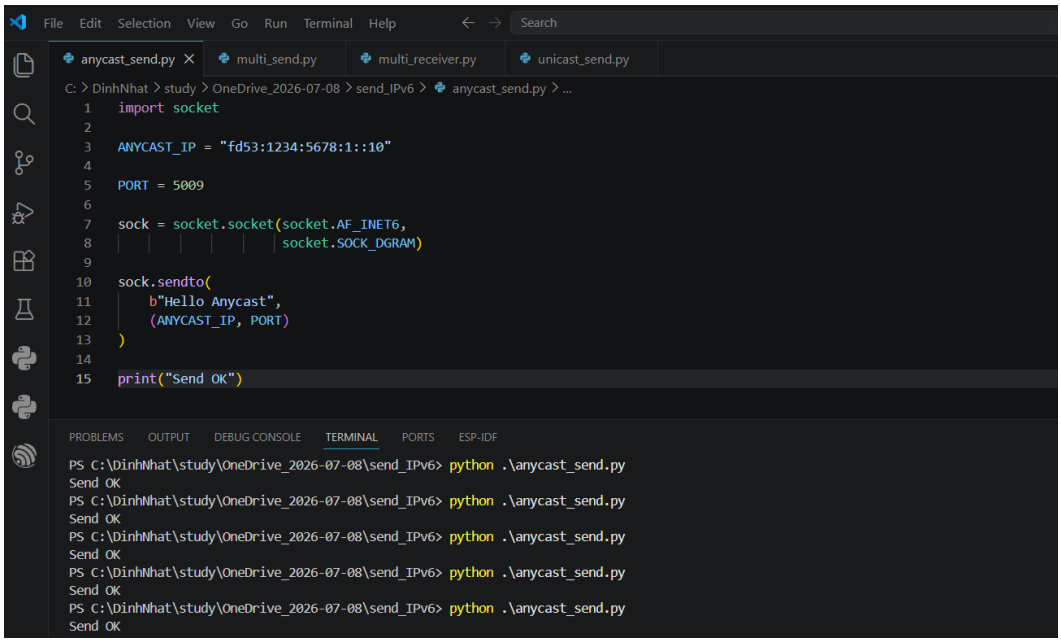
Raspberry Pi nhận Anycast từ Windows 1 (`fd53:1234:5678:1::80`) thông qua địa chỉ dùng chung `fd53:1234:5678:1::100`. Rule mở cổng UDP 5009 ghi nhận 29 gói đi qua thành công, xác nhận Anycast được chấp nhận đúng theo whitelist cổng dịch vụ.



Hình 52: Raspberry Pi nhận IPv6 Anycast qua cổng UDP 5009.



Hình 53: Bộ đếm tường lửa ghi nhận 29 gói Anycast được chấp nhận.



Hình 54: Windows 1 gửi IPv6 Anycast sau khi áp dụng rule.

IPv6_Multicast_Receiver.py anycast_receiver.py

anycast_receiver.py > ...

```python
import socket

PORT = 5009

sock = socket.socket(socket.AF_INET6,
                     socket.SOCK_DGRAM)

sock.bind(("::", PORT))

print("Anycast Server Listening...")

while True:
    data, addr = sock.recvfrom(1024)

    print("Receive:", data.decode())
    print("From:", addr[0])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
Receive: hehehehehehehhehe
From: fd53:1234:5678:1::80
```

Hình 55: Windows 2 nhận lưu lượng Anycast tại địa chỉ dùng chung.

5.3 Bảng tổng hợp so sánh

Bảng 2: So sánh kết quả trước và sau khi áp dụng rule tường lửa.

Phương thức	Trước rule (baseline)	Sau rule (firewall)	Nguyên nhân
IPv4 Unicast	Nhận thành công	Bị drop 10/10 gói	Không nằm trong whitelist cổng/địa chỉ
IPv4 Broadcast	Mọi máy đều nhận	Được accept qua cổng 5008	Cổng dịch vụ nằm trong whitelist
IPv4 Multicast	Nhận theo đúng nhóm đã tham gia	Nhận nhóm `224.12.1.1`; không nhận nhóm `224.1.1.1` khi chưa tham gia	IGMP group membership kết hợp whitelist cổng 5007
IPv6 Unicast	Nhận thành công	Bị drop	Không nằm trong whitelist
IPv6 Multicast	Nhận theo đúng nhóm đã tham gia	Nhóm `ff15::200` bị drop do rule chỉ cho `ff15::100`	Whitelist theo nhóm địa chỉ cụ thể
IPv6 Anycast	Nhận thành công	Được accept qua UDP 5009; bộ đếm ghi nhận 29 gói	Cổng dịch vụ nằm trong whitelist

6 Đánh giá hiệu năng và an toàn

Từ kết quả thực nghiệm, có thể rút ra các nhận định sau:

- **Unicast** có đích xác định và dễ áp dụng whitelist theo địa chỉ/cổng nên thuận lợi cho kiểm soát truy cập. Tuy nhiên, khi cùng một dữ liệu cần gửi đến nhiều máy, Unicast phải tạo nhiều luồng riêng và kém hiệu quả về băng thông. Phương thức này phù hợp với lệnh điều khiển hoặc cấu hình một–một.
- **Broadcast** tạo tải mạng lớn nhất và có rủi ro bảo mật cao vì mọi thiết bị trong miền quảng bá đều nhận gói. Broadcast chỉ nên dùng cho discovery hoặc DHCP trong mạng nội bộ tin cậy và cần được giới hạn chặt theo cổng dịch vụ.
- **Multicast** cân bằng giữa hiệu năng và khả năng kiểm soát. Phương thức này giảm tải so với Broadcast vì chỉ gửi đến nhóm đã đăng ký qua IGMP/MLD; tường lửa có thể bổ sung một lớp kiểm soát bằng cách whitelist đúng nhóm địa chỉ. Multicast phù hợp với streaming và thông báo nhóm.
- **Anycast** tận dụng cơ chế định tuyến để chọn điểm có đường đi tối ưu, giúp giảm độ trễ và tăng tính sẵn sàng nhưng đòi hỏi hạ tầng định tuyến phù hợp. Trong mạng LAN thực hành đơn giản, Anycast hoạt động gần giống Unicast nhưng vẫn thể hiện khả năng dùng chung một địa chỉ cho nhiều endpoint.
- Việc kết hợp group membership (IGMP/MLD) với rule tường lửa theo cổng và địa chỉ tạo mô hình phòng thủ theo lớp (defense in depth). Nếu một lớp được cấu hình chưa đúng, lớp còn lại vẫn có thể ngăn lưu lượng trái phép.

7 Kết luận

Bài thực hành tuần 3 đã khảo sát bốn phương thức truyền thông Unicast, Broadcast, Multicast và Anycast trên IPv4/IPv6, đồng thời kiểm chứng vai trò của tường lửa trong việc kiểm soát từng loại lưu lượng theo cổng dịch vụ và nhóm địa chỉ.

Kết quả đối chứng trước và sau khi áp dụng rule cho thấy hệ thống hoạt động đúng thiết kế: Unicast bị chặn khi không nằm trong whitelist; Broadcast, Multicast và Anycast chỉ được phép đi qua khi thỏa mãn các điều kiện về nhóm địa chỉ và cổng dịch vụ. Đây là cơ sở thực nghiệm để lựa chọn phương thức truyền và xây dựng chính sách tường lửa phù hợp cho các hệ thống mạng thực tế.

Tài liệu tham khảo

[1] Báo cáo thực tập tuần 2 – LG DUT Cyber Security Lab (VLAN, IPv6, Scapy, nftables).
[2] Tài liệu đào tạo “Firewall” – LGEDV–DUT Internship Training Collaboration Program (IPv4/IPv6, Unicast/Multicast/Broadcast).
[3] Ghi chú thực nghiệm “Vai trò gốc” và “Sau khi chạy rule firewall” (tài liệu nội bộ, tuần 3).
[4] GeeksforGeeks, “Difference between Unicast, Broadcast and Multicast in Computer Network”, https://www.geeksforgeeks.org/difference-between-unicast-broadcast-and-multicast-in-computer-network/.
[5] GeeksforGeeks, “Differences between IPv4 and IPv6”, https://www.geeksforgeeks.org/differences-between-ipv4-and-ipv6/.